LayerTree: Single-Transaction Exit and Eviction from a Shared Bitcoin UTXO

Olaoluwa Osuntokun

Lightning Labs
laolu@lightning.engineering

Abstract. Constructions that let many users share one Bitcoin output pay at exit time: a user who leaves publishes $O(\log N)$ transactions, the tree's interior must exist as pre-signed or template-committed transactions, and one unresponsive user forces the whole tree on-chain at expiry. We present LayerTree, in which a single output commits to an off-chain Merkle-sum tree of balances and tapscript covenants enforce every state transition. Any member exits, and the operator evicts any expired member, in *one* transaction whose size grows with the depth of the tree (about 32 virtual bytes per level) rather than with a path of transactions: 1,020 virtual bytes at depth 16. Under a simple model of an Ark- or CTV-style transaction tree of the same depth this is about 60% smaller; under mass exit, where the transaction tree's interior is shared, the gap narrows. Membership changes need no interactive signing round, the shared output never expires, and whole cohorts are added or swept in one constant-shape transaction (158 to 203 virtual bytes per evicted leaf for cohorts of 51 and 13).
The construction composes only opcodes already proposed for Bitcoin: the restored string and arithmetic opcodes of tapleaf version `0xC2`, transaction introspection, signature checks over stack messages and elliptic-curve tweaks. We realise every transition against the full candidate set, all but one against a `0xC0`-only subset, and the core exit against `OP_CAT` plus one point-tweak opcode alone, by having the script rebuild the BIP-341 signature message and check it with a generator-key signature. Lightning channels are hosted inside leaves with prevout-independent commitments, so a hub of N channels occupies one output. We prove conservation unconditionally, and proof soundness and payability under a real-tree invariant, in the idealised hash model, mechanised in Lean 4. A Python simulator, a byte-exact Go implementation and 338 executable checks accompany the design.

Keywords: Bitcoin · covenants · Merkle-sum trees · payment channels · layer two · formal verification

1 Introduction

Payment channels [26] move Bitcoin activity off-chain for pairs; the open problem is doing it for *groups*: letting N users share one unspent transaction output

(UTXO) without giving up the ability of each to leave alone. Every deployed or proposed approach pays for the sharing when it is dissolved. In channel factories [3], CoinPool [27], Ark [13,14], timeout trees [15,38] and covenant trees built on `OP_CHECKTEMPLATEVERIFY` [28] or `OP_CHECKCONTRACTVERIFY` [12,11], the shared output is the root of a tree of *transactions*. A member who leaves unilaterally must publish the path from the root to its leaf, $O(\log N)$ transactions each paying a fee; the interior of the tree must exist in advance as pre-signed transactions (requiring every member to sign, and to be online) or as covenant templates; and when the tree expires, reclaiming the funds of a single unresponsive member means publishing everything.

This paper takes a different route. The tree stays off-chain and is a tree of *data*, not transactions: a binary Merkle-sum tree of balances whose root is committed by one output. A member exits by presenting its leaf and sibling path to the output itself; the spending script recomputes the root from the proof, checks it against the commitment, recomputes the root with the leaf removed, and requires the transaction to recreate the output under the new root with the leaf's amount paid out. The member needs nobody's signature, the exit is one transaction, and its size grows with the tree's depth (about 32 virtual bytes per level) rather than with a path of transactions. The same covenant lets the operator evict expired members, graft a pre-built subtree of new members in one step, and sweep an expired cohort cheaply.

This needs script capabilities that are proposed for Bitcoin but not yet consensus: the restored string and arithmetic opcodes of a new tapleaf version (BIPs 440 and 441 [30,31]), transaction introspection (`OP_TX` [29]), signature checks over stack data (`OP_CHECKSIGFROMSTACK` [1]) and elliptic-curve arithmetic [24,4]; several already exist on Liquid [2]. We propose no opcode of our own, and part of the contribution is a measured account of how little of the candidate set is actually needed.

Contributions.

1. **Construction** (§4). A shared output committing to a Merkle-sum tree through a data-tweaked taproot internal key, with transitions for unilateral exit, eviction, batched exit, subtree grafting, cooperative update, single-use vouchers and cohort sweeps, in a dense operator-assigned profile and a key-addressed sparse profile with compressed proofs.
2. **Realisation in three opcode tiers** (§5). Every transition is implemented against the full candidate set; every transition but the cohort sweep against a `0xC0`-only subset; and the single-leaf exit, the load-bearing case, against `OP_CAT` and one point-tweak opcode alone. The last tier replaces introspection by a constructive trick, rebuilding the BIP-341 signature message in script and checking a signature under the generator point. This gives an *upper bound* on the opcodes the covenant needs; we prove no lower bound.
3. **Channel leaves** (§6). Lightning channels hosted in leaves, with prevout-independent commitments that emulate `ANYPREVOUT` using introspection and stack signature checks, and an update/settlement state machine, so a funding

output need not exist until a dispute; and the three tree-layer rules shared-key leaves require.

4. **Security analysis with mechanised proofs** (§7). Conservation, proof soundness and payability in the idealised hash model, checked in Lean 4. The proofs isolated a fact the informal argument had blurred: sums are conserved unconditionally, but exits are guaranteed only under a real-tree invariant that creation and grafting must establish off-chain.
5. **Implementation and measurement** (§8). A tapscript simulator for the candidate opcodes, a byte-exact Go implementation, and measured costs: a 1,020 vB exit at depth 16, a 665 vB marginal batched exit, cohort eviction at 158 to 203 vB per leaf, and fees from 1 to 250 sat/vB.

The specification, code and proofs are public [25].

2 Background

A pay-to-taproot output [35] commits to an internal key P and the Merkle root m of a *taptree* of scripts through $Q = P + H_{\mathsf{TapTweak}}(P \,\|\, m) \cdot G$, and is spent either by a Schnorr signature [34] under Q or by revealing one script with its Merkle path (the *control block*) and satisfying it. Tapscript [36] has no loops, so every "loop" below is unrolled and costs bytes per iteration. BIP-441 [31] defines tapleaf version `0xC2`, restoring the string opcodes disabled in 2010 (`OP_CAT`, `OP_SUBSTR`, `OP_LEFT`, `OP_RIGHT`, bit operations) with arithmetic on unsigned numbers of any length, metered by BIP-440 [30] ("varops", 10,000 units per weight unit of the transaction); BIP-347 [10] proposes `OP_CAT` alone in the current version. `OP_TX` [29] pushes selected fields of the spending transaction, optionally collated into one hash preimage; `OP_CHECKSIGFROMSTACK` [1] verifies a Schnorr signature over a stack element; `OP_TWEAKADD` [4] computes $x(P+tG)$, and the EC-ops draft [24] adds point addition, scalar multiplication and negation. All are proposals; we note where the design depends on open details. A Merkle-sum tree [16,23] hashes each node's children together with their sums, so an inclusion proof proves a leaf's contribution to the root sum; sparse Merkle trees [5] address positions by hash, making non-membership an inclusion proof of an empty leaf.

3 Model and Goals

A *tree* has one *operator* and any number of *members*. Each member owns a *leaf*: a balance and three public keys (an owner key that authorises the member's actions, an exit key paid on exit, and an evictor key, normally the operator's, that may remove the leaf after a lease height). The operator holds no key over the funds, only an *operator key* that authorises additions and co-signs cooperative changes. The *tree output* is the on-chain UTXO; its amount equals the sum committed by the root and its script commits to the root.

We assume the Bitcoin consensus rules with the opcodes of §2 active, collision-resistant tagged hashing (SHA-256), the discrete-logarithm assumption on secp256k1,

and unforgeable Schnorr signatures. The adversary controls the operator, any subset of members, and the order of transactions within the rules; it may refuse to serve data. We want:

G1 Conservation. No transition creates value: the output's amount equals the sum committed by its root, and payouts are funded by removed leaf sums. Under the real-tree invariant of §7 that sum is the members' balances.
G2 Unilateral single-transaction exit. A member holding its leaf and path can construct an accepted exit paying its balance in one transaction of $O(D)$ bytes with no other party's signature, given a fee source, a balance above dust, and the contention bound of §7.
G3 Eviction. After a leaf's lease height its evictor can remove it, paying the balance to the exit key.
G4 Non-interactive membership. Adding or removing members needs no signing round among members and no pre-signed transactions.
G5 No expiry. The shared output has no lifetime; leases are per leaf and renewable.

G1 and G2 are the subject of the theorems of §7; G3 to G5 hold by construction (eviction is an authorisation branch of the exit script; no transition needs a signature but the actor's; no script carries an expiry) and are outside the verified perimeter. Liveness of joins is not a goal, and data availability is a member's responsibility.

4 Construction

4.1 The Tree

We write $\mathsf{le}_k(x)$ for the k-byte little-endian encoding of x. Fix a depth D. A tree has 2^D positions, each empty or holding a leaf. A leaf is a fixed 116-byte record:

$$\mathsf{le}_{64}(\mathsf{amount}) \,\|\, \mathsf{exit_key} \,\|\, \mathsf{owner_key} \,\|\, \mathsf{evictor_key} \,\|\, \mathsf{le}_{32}(\mathsf{evict_after}) \,\|\, \mathsf{le}_{32}(\mathsf{nonce}) \,\|\, \mathsf{le}_{32}(\mathsf{seq}),$$

with keys as 32-byte x-only points, $\mathsf{evict_after}$ a block height, nonce a per-leaf identifier and seq a counter advanced by vouchers (Appendix D.1). A *node* is a pair (h, s) of a 32-byte hash and an unsigned sum. A position holding leaf ℓ has node $(H_{\mathsf{leaf}}(\ell), \ell.\mathsf{amount})$; an empty position has node $E_0 = (0^{32}, 0)$. The parent of (h_L, s_L) and (h_R, s_R) is

$$\big(H_{\mathsf{node}}(h_L \,\|\, \mathsf{enc}_{64}(s_L) \,\|\, h_R \,\|\, \mathsf{enc}_{64}(s_R)),\ s_L + s_R\big), \qquad \mathsf{enc}_{64}(s) = \mathsf{le}_{64}(s) \,\|\, \mathtt{0x01}.$$

Every hash in this paper is a BIP-340 tagged hash with its own tag, so no preimage of one is a preimage of another. $\mathsf{enc}_{64}(s)$ is chosen so that a script produces it from a computed sum with one addition: the minimal BIP-441 encoding of $s + 2^{64}$ is $\mathsf{le}_{64}(s) \,\|\, \mathtt{01}$ (Lemma 1). We write E_h for the node of the empty tree of height h; these constants are hard-coded in scripts, and E_D is the empty tree's root.

The *sibling path* π of a position is the list of D pairs $(\mathsf{dir}_k, (h_k, s_k))$, one per level from the root down, giving the direction the position lies on (R if the node being proved is the right child) and the sibling node not taken. $\mathsf{fold}(n, \pi)$ combines a node n with a path level by level from the bottom up: at each level it hashes n with the sibling in the order the direction gives, with the two sums, and adds the sums. The witness of §5 supplies the same pairs deepest level first, because the script consumes them in that order. The two facts that drive the design are that the honest path folds a leaf's node to the root, and that folding a *different* node up the *same* path yields the root of the tree with that position replaced (§7).

Profiles. In the *dense* profile positions are assigned by the operator and a path carries explicit direction bits; each level costs 88 bytes of script and 41 bytes of witness. In the *sparse* profile a leaf's position is the low D bits of $H_{\mathsf{pos}}(\mathsf{owner_key} \,\|\, \mathsf{nonce})$, direction bits are derived in script from the position hash, and empty siblings are elided from the witness (a one-byte marker in place of a node; the script substitutes E_k), so witness data grows with $\log_2 N$ for N members rather than with D. The sparse profile is the Merkle-sum sparse Merkle tree of Taproot Assets [23] truncated to $D = 32$ or 64: key-addressed positions, a history-independent root and structural uniqueness of leaves, at 1.8 to 2.7 times the per-exit cost of the dense profile at comparable membership (Table 2).

4.2 The Tree Output

Let r be the root hash and S the root sum. The tree output is a taproot output with internal key

$$P = \mathsf{lift_x}\big(x(\mathsf{NUMS} + H_{\mathsf{state}}(r) \cdot G)\big),$$

where NUMS is the BIP-341 nothing-up-my-sleeve point and $\mathsf{lift_x}$ the BIP-340 even-y normalisation, so the key path is unspendable and every spend goes through a script. Its taptree contains one leaf per supported transition script; the taptree's Merkle root m identifies the rule set and is constant for the tree's life. We write $\mathsf{TreeSPK}(r, m)$ for the resulting scriptPubKey. A tree output for state (r, S) is an output with script $\mathsf{TreeSPK}(r, m)$ and amount S.

Every transition script receives r and m in its witness, recomputes $\mathsf{TreeSPK}(r, m)$ and checks it against the spent input's scriptPubKey, read with `OP_TX`. This binds both the root the proofs are checked against and the script set the successor carries; the scripts cannot hard-code m since m depends on them, and binding it is security critical, or a spender could recreate the tree under scripts of its choosing. Since every script checks input 0, two trees cannot be merged or spent atomically.

4.3 Transitions

Every transition spends the tree output as input 0 and recreates it as output 0 with the new root and the new total; payouts, if any, occupy outputs $1, \ldots, K$;

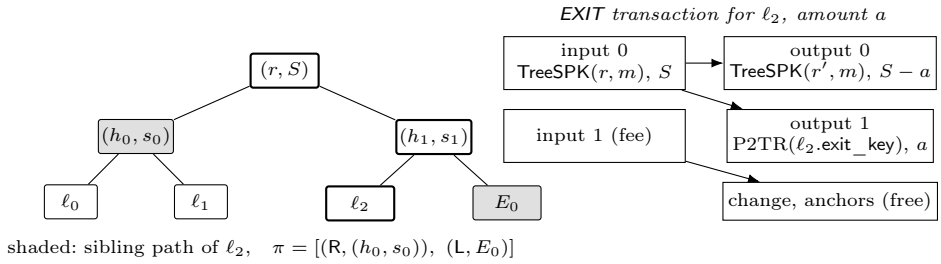


Fig. 1. A depth-2 tree and the exit of ℓ_2. The script checks $\mathsf{fold}((H_{\mathsf{leaf}}(\ell_2), a), \pi) = (r, S)$, computes $r' = \mathsf{fold}(E_0, \pi)$, verifies a signature by ℓ_2.owner_key, and constrains outputs 0 and 1. Everything else in the transaction is free.

all other inputs and outputs are unconstrained, so fees, change and anchors are free. The common core of every transition is the *step*: given a path π, an old node and a new node, check that the old node folds to the committed root (r, S), with S read as input 0's amount (CHECK_OLD), and compute the root the new node folds to; a transition is a sequence of steps, each proving its old node against the root the previous one produced (CHECK_PREV), followed by a check that output 0 is $(S', \mathsf{TreeSPK}(r', m))$ unless $r' = E_D$ (FINAL). Appendix B gives the pseudocode and the 88-byte tapscript of one level; Appendix E lists every script of every tier in full.

EXIT. Removes one leaf ℓ: the old node is $(H_{\mathsf{leaf}}(\ell), \ell.\mathsf{amount})$, the new node E_0, and output 1 must pay P2TR(ℓ.exit_key). A witness flag selects the authorisation. An *owner exit* needs a signature by ℓ.owner_key over the transaction and, in the FULL tier, permits output 1 to be at most ℓ.amount (the owner consents to the slack as fee by signing). An *eviction* needs a signature by ℓ.evictor_key, a lock time of at least ℓ.evict_after, and output 1 *equal* to ℓ.amount: an evictor may remove but never shortchange. Signatures must cover all outputs, or a relayer could redirect the slack. The exact-payout variant $\mathsf{EXIT}^{=}$ requires equality for owner exits too; it is a tree-creation parameter, the only form in MID and MIN, and matters for shared-key leaves (§6).

JOIN$_h$. Grafts a subtree of height h into an empty aligned slot: the old node is the constant E_h, the new node $(h_{\mathrm{new}}, s_{\mathrm{new}})$ is supplied and signed by the operator, and output 0 rises by s_{new}, funded by other inputs. With $h = 8$ this adds up to 256 members for a constant script cost. The covenant cannot verify the grafted interior; prospective members verify their own paths inside it before funding (§7). JOINK_K sequences K single-leaf joins under one operator signature, the batching primitive of the sparse profile, where grafting is unavailable.

UPDATE. Replaces one leaf by another at the same position with signatures from both the owner and the operator; the output amount moves by the difference. This is the splice, the lease renewal and the ownership transfer. A variant UPDATE^{x}

Table 1. The three opcode tiers. Sizes are bytes of script / witness data for EXIT_1 in the dense profile at $D = 16$; the last row is the transaction size.

	FULL	**MID**	**MIN**
Tapleaf version	0xC2	0xC0	0xC0
Beyond BIP-342	OP_CAT, splice, bit ops, OP_TX, CSFS, EC ops	OP_CAT, OP_TX, CSFS, OP_TWEAKADD	OP_CAT, OP_TWEAKADD
Introspection	OP_TX	OP_TX	constructive (§5.3)
Numerics, sum cap	any length, none	4-byte, 2^{31} sat	4-byte, 2^{31} sat
Owner payout	$\leq$ amount (fee option)	exact	exact
Transitions realised	all, both profiles	all but $\mathsf{EVICT_SUBTREE}_h$	EXIT_1, dense
Executable checks	272	59	7
Vouchers / channels / tx shape	yes / yes / free	yes / yes / free	no / no / fixed
EXIT_1 script / witness (B)	2,101 / 902	3,299 / 978	3,572 / 1,175
EXIT_1 transaction (vB)	1,020	1,338	1,456

additionally requires a signature by the *old* leaf's exit key, for leaves whose exit key is shared with a third party (§6).

CLAIM. Executes a *voucher*, an off-chain owner-signed instruction to carve a new leaf out of an existing one, verified with OP_CHECKSIGFROMSTACK; the total is unchanged and replay is impossible because the first of its two steps changes the source leaf. Vouchers are optional and orthogonal to the rest of the design; Appendix D.1 specifies them.

Composition. EXIT_K applies K exits in one transaction, each with its own authorisation, paying outputs $1..K$; the k-th step proves its leaf against the root step $k-1$ produced. The exiters must agree on the transaction, but nothing is pre-signed and no one outside the batch is involved. $\mathsf{EVICT_SUBTREE}_h$ is a *shape-specialised multiproof*: one sibling path to an aligned subtree of height h and the 2^h leaves below it (or empty markers); the script recomputes the subtree root in post-order, checks each leaf's evictor key and expired lease, pays each leaf to its exit key at the next unused output, and replaces the subtree by E_h, hashing the interior once rather than once per leaf. A flag-driven generic multiproof was measured and rejected: without loops it saves no script. If the new root is E_D the tree is empty and output 0 is unconstrained, so a tree dissolves without leaving a zero-value output.

5 Script Realisation and Opcode Tiers

The transitions are realised against three opcode sets (Table 1), so that the design's dependence on each candidate opcode is measured rather than asserted; the tree format is the same in every tier. Coverage is not uniform: FULL implements every transition in both profiles, MID everything but the cohort sweep, and MIN the dense single-leaf exit only, the other transitions following the same construction but being argued, not measured.

5.1 The FULL Tier

With `0xC2` the leaf is one 116-byte stack element read with `OP_SUBSTR`; sums use arbitrary-length arithmetic and are padded with $\langle 2^{64} \rangle$ `OP_ADD` (Lemma 1); one LEVEL of the fold is 88 bytes (Appendix B). TreeSPK is recomputed with the EC opcodes ($H_{\mathsf{state}}(r) \cdot G + \mathsf{NUMS}$, tweaked by the taptree hash, wrapped in a P2TR script). The EXIT_1 prologue, with both introspections and the key recomputation, is under 700 bytes.

5.2 The MID Tier

Tapleaf `0xC0` keeps BIP-342 semantics: no splice opcodes and 4-byte signed numbers. Fields arrive as separate witness elements, each checked with `OP_SIZE` and re-joined with `OP_CAT` before hashing, and a computed sum is widened to 8 bytes by a witness-supplied zero pad checked with `OP_SIZE` and `OP_CAT` (PADZ, Appendix B); `MINIMALDATA` policy means lock times and positions arrive as minimal numbers plus a pad. A cap of 2^{31} satoshis per tree and exact payouts are the price of 4-byte arithmetic. The tier assumes `OP_TX` in `0xC0`, which the draft leaves open; Liquid [2] has per-field introspection in the same role today.

5.3 The MIN Tier: Introspection from a Signature

The covenant must read the input's scriptPubKey and amount and outputs 0 and 1. Without `OP_TX` the only opcode that sees the transaction is `OP_CHECKSIG`, and we turn it into introspection. The script *constructs* the BIP-341 signature message from values it has derived (the recomputed $\mathsf{TreeSPK}(r, m)$, the input amount, the new tree output, the payout) and raw witness pieces for everything it does not constrain (the fee input, prevouts, sequences, version, lock time, tapleaf hash), hashes it, and demands a Schnorr signature under the generator G with nonce $R = G$. For public key G and nonce G the challenge is $e = H_{\mathsf{challenge}}(G_x \,\|\, G_x \,\|\, \mathsf{msg})$ and the verification equation $s \cdot G = R + e \cdot P = (1+e) \cdot G$ has the unique solution $s = e + 1 \bmod n$; G has even y, so $\mathsf{lift_x}(G_x) = G$ on both sides and the even-y check on R passes. The script computes e from its constructed message, forms s, and calls `OP_CHECKSIG` with the signature $G_x \,\|\, s$ and public key G_x. Since `OP_CHECKSIG` hashes the real transaction, the check passes only if the constructed message equals the real one, up to a collision in the tagged hashes; every field the script fixed is thereby enforced.

Three caveats. The "signature" authorises nothing (anyone can produce it; authorisation is the separate owner or evictor `OP_CHECKSIGVERIFY`); what it adds is binding, resting on collision resistance of the TapSighash and challenge hashes, an assumption outside the Lean model of §7. Without arbitrary-precision arithmetic, $e+1$ is formed by grinding: the spender varies the fee input's sequence number (within the range that neither imposes a BIP-68 lock nor disables BIP-125 signalling) until the last byte of e is `00` and supplies the first 31 bytes; the script checks $e_{[0,31)} \,\|\, \mathtt{00} = e$ and uses $e_{[0,31)} \,\|\, \mathtt{01}$ as s. This takes 256 trials on average, must precede signing the fee input, and makes fee bumping a matter of a child

spending a free output rather than replacement. In the 2^{-128} event that $e \geq n-1$ the signature does not parse and the spender re-grinds. The script guarantees that input 0 is a tree output with amount S and script $\mathsf{TreeSPK}(r, m)$, that there are exactly two inputs, that input 0 carries no annex, and that outputs 0 and 1 are as the covenant requires; everything else is free but must be presented consistently (Appendix B).

The transaction shape is fixed per script, the tree output key is recomputed with two `OP_TWEAKADD`s, and EXIT_1 costs 3,572 bytes of script at $D = 16$ (70% more than FULL) for a transaction of 1,456 vB (43% more). Vouchers are impossible without a signature over stack data, and so are channel leaves. The result is an upper bound: BIP-342 tapscript, `OP_CAT` and one point-tweak opcode realise the exit covenant. Every restored splice and bit opcode, `OP_TX` and the EC point opcodes are conveniences: for the dense exit at $D = 16$, dropping the splice and EC opcodes costs 260 witness units and dropping `OP_TX` as well 1,744. Whether the largest scripts (a 64-leaf sweep is 16 kB in FULL) fit MIN's constraints is open; we claim no lower bound.

6 Channel Leaves

A leaf whose exit key is a 2-of-2 between its owner and a counterparty (normally the operator) can host a payment channel that exists only off-chain and whose funding output materialises at the payout of the transaction that removes the leaf: a tree of N such leaves is a Lightning hub behind one UTXO, with opening, resizing and closing costing nothing on-chain except at dispute. Appendix C gives scripts, state machine and costs.

Rebindable commitments. The funding outpoint is unknown when channel states are signed, and it changes with every tree transition. Commitment transactions must therefore be authorised by signatures that do not commit to the prevout, which `OP_TX` and `OP_CHECKSIGFROMSTACK` provide as an emulation of `SIGHASH_ANYPREVOUT` [7]: the script collates, with one `OP_TX` call, the fields the signature covers (version, lock time, the current input's sequence, the number of inputs, and all outputs), prefixes a per-channel context constant baked into the script, hashes, and verifies a BIP-340 signature under the channel key with `OP_CHECKSIGFROMSTACK`. The context commits to the tree identifier m, the leaf identity, the channel key, the capacity and a splice epoch, so a state signed for one channel or capacity never verifies for another; committing the number of inputs (one) prevents two funding outputs with the same script from being spent by one commitment. The input amount is not committed by the update signature: the committed outputs sum to the capacity, so an input holding more would only burn the excess to fees, and rule (i) below makes it hold exactly the capacity.

State machine. Penalty channels cannot be used, since a penalty transaction must be pre-signed against a specific outpoint. Channel leaves use the update/settlement ratchet of eltoo [6,18]: the update transaction for state n carries lock

time $S_0 + n$ (S_0 the start of the timestamp range) and spends the funding output or any earlier update output through a leaf admitting only later lock times; the settlement spends state n's update output after a relative delay csv_delay through a leaf whose signature also commits to the spent script, so it binds only to its own state. Both are zero-fee version-3 transactions with an ephemeral anchor [37]. A responder needs the channel context and the counterparty's latest signature but no per-state script, since the update output's taptree is a function of the context and n, which is published in the lock time.

HTLCs and deadlines. Settlement outputs include one per in-flight HTLC, with no second stage since the settlement is itself delayed; payments between channel leaves of one tree are HTLC forwards through the operator and never touch the tree output. The deadline budget an HTLC needs is

$$\mathsf{cltv_expiry_delta} \;\geq\; G_{\text{exit}} + 1 + 1 + \mathsf{csv_delay} + 1 + G_{\text{sweep}},$$

where G_{exit} is the time to get the exit confirmed *including contention for the tree output*, then one block each for the update and settlement transactions, the delay between them, and the HTLC sweep. With $\mathsf{csv_delay} = 144$ and a conservative $G_{\text{exit}} = 72$ this is about 220 blocks, against 40 to 80 on the public network; and G_{exit} grows with the number of members under the contention bound of §7, so the budget is a function of tree size.

Three tree-layer rules. Three tree rules were too lenient for a leaf whose balance is not the owner's alone. (i) The owner's fee-from-payout option would let the owner underfund the channel and hold the counterparty's balance hostage, so channel leaves require exact payouts and their trees should use $\mathsf{EXIT}^{=}$, visible in m. (ii) A voucher lets the owner carve value out of a leaf alone, so channel leaves set seq to the value whose successor does not fit in four bytes and thereby cannot be claimed: `0xFFFFFFFF` under BIP-441 numerics but `0x7FFFFFFF` under `CScriptNum`, where `ff ff ff ff` encodes $-(2^{31} - 1)$ (Lemma 1). (iii) Before the lease height the counterparty can enforce nothing on-chain, so a routing channel leaf sets it to zero. For channels between two members, UPDATE^{x} prevents the owner and operator from splicing the leaf down behind the counterparty's back. Channel leaves need `OP_TX` and `OP_CHECKSIGFROMSTACK`, hence FULL or MID; MIN's constructive introspection necessarily commits to the prevout.

7 Security Analysis

We work in the idealised hash model: H_{leaf} and H_{node} are injective on the domains the protocol uses, and digests under different tags never coincide or equal the zero string; a proof under injectivity holds against any adversary that cannot exhibit a collision. The theorems are stated for the abstract rules of §4.3 and mechanised in Lean 4 [17] in about 1,300 lines without Mathlib (Appendix A); bracketed names are Lean identifiers. The correspondence between rules and bytecode is by execution (§8), not proof.

Lemma 1 (Encodings). *For $s < 2^{64}$, the minimal BIP-441 encoding of $s + 2^{64}$ is $\mathsf{le}_{64}(s) \,\|\, \mathtt{01}$; leaf and node serialisations are injective under the field bounds; the minimal encoding of 2^{32} has five bytes while that of $2^{32} - 1$ has four; and the `CScriptNum` of $-(2^{31} - 1)$ is `ffffffff` with successor `feffffff`. [enc64_eq_minimal, serializeLeaf_inj, serializeNode_inj, minimalLE_two_pow_32_length, scriptNum_neg_max31_succ]*

Theorem 1 (Binding). *Roots bind trees: two trees of the same depth with equal roots are equal. Any node n and path π with $\mathsf{fold}(n, \pi)$ equal to the root of a tree T are the root of one of T's subtrees and that subtree's honest path. In particular the sum in n is the true sum of that subtree, and a leaf node that verifies with a full-length path is the leaf at that position. [root_inj, fold_sound]*

Proof (sketch). Induction on the path. A non-empty path claims an internal node, so the tree is not a tip by tag separation. Injectivity of H_{node} on the parent's preimage equates both children's hashes *and* sums with the claimed ones, and the induction hypothesis applies to the child the direction bit selects. This is the only place the hash axioms are used.

Theorem 2 (Conservation). *For any sequence of valid steps from root (r, S) to (r', S'), $S' + \sum s_{\text{old}} = S + \sum s_{\text{new}}$. Hence for EXIT_K, $S' + \sum_k \text{payout}_k \le S$, with equality under exact payouts; for JOIN_h, $S' = S + s_{\text{new}}$; for UPDATE, $S' + a_{\text{old}} = S + a_{\text{new}}$; and CLAIM leaves S unchanged. [Steps.sum, ExitK.no_inflation, ExitK.exact_conservation, join_sound, update_sound, claim_sound]*

Proof (sketch). A fold adds exactly the sibling sums it is shown, so the two folds of a step differ by $s_{\text{new}} - s_{\text{old}}$; the rest is induction on the sequence. No hash assumption is needed: the covenant conserves sums whatever nodes it is shown.

Theorem 3 (Payability). *For any tree T and position p holding leaf ℓ, the honest witness (the leaf, its path, E_0) is a valid step from T's root to the root of T with p emptied, the sum falls by $\ell.\mathsf{amount}$, and every other position is unchanged. [payable]*

Theorem 4 (Real-tree invariant). *If the current root is the root of a tree and every step is well formed and* grounded *(its new node is the root of some tree of the step's height), the final root is the root of a tree of the same depth. Leaf-level steps are grounded by construction. [Steps.sound]*

Establishing and maintaining the invariant. Theorems 1 and 3 are stated for a tree, while the on-chain object is a root. The invariant that the committed root is the root of a real tree whose total is the output amount has a base case, creation, which the covenant cannot enforce: a founding member must verify its path against r_0 and the funding amount before contributing (a wrong amount yields an output no script can spend, never a theft). Its step is Theorem 4: every transition but a JOIN graft is grounded by construction, and a graft is grounded exactly when the operator's $(h_{\text{new}}, s_{\text{new}})$ has a preimage tree. If it does not, Theorem 2 still holds, so no value is created, but nothing below the

node can ever be proven: the grafted value is frozen. The covenant protects the *sum*; members protect their *membership* by verifying their paths inside a subtree before funding a join, which the specification makes mandatory. Under the invariant, Theorems 2 and 3 give G1 and G2: whoever holds a leaf and its path can construct an accepted exit paying the leaf amount without anyone's cooperation or touching anyone else's leaf. The mechanisation also showed that CLAIM requires the claim position to be empty *after* the source step, so a drained source may serve as the claim position; this is harmless.

Root binding to the output. The scripts conclude that the witness root and taptree are the committed ones from $\mathsf{TreeSPK}(r', m') = \mathsf{TreeSPK}(r, m)$, which needs TreeSPK injective in (r, m) [witness_root_binds]; Appendix A reduces this to the discrete logarithm of NUMS and collisions in H_{state} or the taproot tweak hash. Without binding m a spender could recreate the tree under scripts of its own.

Authorisation and lock times. These decide who may perform a transition, not what it does to the tree, and the theorems carry them as opaque propositions; G3 to G5 rest on them. The requirement that exit signatures cover all outputs matters because the FULL covenant only bounds output 1's amount for owner exits; with `SIGHASH_NONE` a relayer could divert the slack.

Concurrency and pinning. All transitions spend one UTXO, so unconfirmed transitions form a descendant chain of the last confirmed tree output. Under today's default policy a package holds at most 25 transactions [9], about a block's worth of transitions per block; cluster mempool [33] changes the bound, not the serialisation. The bound applies to joins and sweeps as well as exits, and the chain invites pinning: a low-feerate exit early in it caps the feerate of everything behind it, and replacing it means paying for its descendants. Mitigations are cooperative batching, anchors so either party can bump, and sizing trees by exit throughput rather than depth: 10,000 channel leaves need about 400 blocks to force-close while HTLC deadlines run. This is the design's honest weak spot.

Data availability and reorganisations. Members retain their leaf and path and process every spend of the tree output; since each witness contains the full sibling path it used, a member updates its path from the chain alone and never depends on the operator to exit. A reorganisation that drops a transition invalidates paths derived from it, so members treat transitions as final only at their confirmation depth.

8 Evaluation

Implementation. A Python tapscript interpreter for both leaf versions models BIP-441 numerics, `OP_TX`, `OP_CHECKSIGFROMSTACK`, the EC opcodes and varops accounting, computes the real BIP-341 signature message and enforces

Table 2. Measured EXIT_1 cost (FULL). WU include a 129-byte control block for a five-leaf taptree (each doubling of the script set adds 8 vB); the transaction adds a fee input, change and overhead (≈ 950 WU). Sparse witness is $326 + 41 \log_2 N$ bytes, tabulated at $N = 2^{16}$. Last column: unamortised radix-2 transaction tree of 2^D leaves, $150D + 111$ vB; sparse D is address width.

profile	D	script (B)	witness (B)	total (WU)	tx (vB)	radix-2 tx tree (vB)
dense	8	1,397	574	2,100	762	1,311
dense	12	1,749	738	2,616	891	1,911
dense	16	2,101	902	3,132	1,020	2,511
dense	20	2,453	1,066	3,648	1,149	3,111
sparse	32	5,147	$326 + 41 \log_2 N$	$\approx$ 6,260	$\approx$ 1,800	n/a
sparse	64	9,499	$326 + 41 \log_2 N$	$\approx$ 10,610	$\approx$ 2,890	n/a
MID dense	16	3,299	978	4,406	1,338	
MIN dense	16	3,572	1,175	4,876	1,456	

`MINIMALDATA`. Every implemented script runs against a reference tree with positive and negative cases per rule (forged leaf, stale root, attacker-chosen m, unconserved amount, underfunded join, early eviction, evictor shortfall, unsigned update, voucher replay, over-claim, stale second path in a batch): 272 FULL, 59 MID and 7 MIN checks, failing if any script uses an opcode outside the candidate set. A Go implementation on `btcd` and `lnd` generates the same scripts, byte-exact against 30 golden vectors, and its BIP-341 signature hash equals the simulator's, which makes the MIN tier testable end to end.

Exit cost. In the dense profile (Table 2, Figure 2) a unilateral exit costs 762 vB at $D = 8$ and 1,149 vB at $D = 20$: a 4,096-fold increase in tree size raises it by 51%, about 32 vB per level. Tier matters more than depth: MID costs 31% and MIN 43% more than FULL. The sparse profile at $D = 32$ with 2^{16} members costs about 1.8 times the dense profile at $D = 16$; its witness grows with $\log_2 N$, not D.

Comparison with transaction trees. The last column of Table 2 is a *model*: a radix-2 tree of 2^D leaves in which every exiter publishes D transactions of about 150 vB and a 111 vB claim. Under it the constructions cross near $D = 3$ and LayerTree is about 60% smaller at $D = 16$. Two corrections work against LayerTree: a radix-4 tree halves the levels at about 220 vB each (roughly 1,870 vB at 65,536 leaves, a 45% gap), and the interior of a transaction tree is shared, so under mass exit its per-member cost falls toward one transaction, below LayerTree's 665 vB marginal batched exit. The model also prices neither the latency of D sequential confirmations nor LayerTree's larger opcode set. LayerTree wins clearly for isolated exits and eviction, and is comparable or worse for a coordinated mass exit.

Batching, joins and cohorts. A second exit in the same transaction costs 665 vB marginally against 1,020 vB alone. A dense JOIN_2 at $D = 16$ is 2,713 WU of witness whatever the grafted subtree holds; onboarding cost is dominated

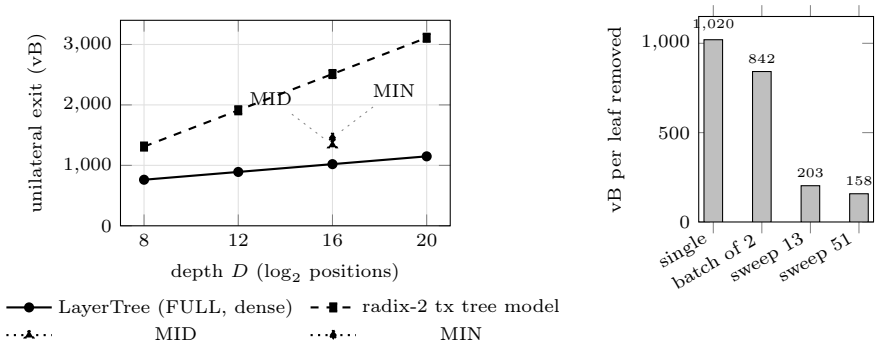


Fig. 2. Left: unilateral exit cost against tree depth, measured for LayerTree and modelled for an unamortised radix-2 transaction tree. Right: per-leaf cost of removal at $D = 16$: a single exit, the average of a two-exit batch, and the cohort sweeps $\mathsf{EVICT_SUBTREE}_4$ (13 leaves) and $\mathsf{EVICT_SUBTREE}_6$ (51 leaves).

by the members' funding inputs (about 58 vB each), which any self-funded construction pays, Ark's round included. Sparse joins cannot graft and cost about 5,000 WU per leaf. Cohort sweeps do for removal what grafting does for addition: $\mathsf{EVICT_SUBTREE}_4$ removes 13 leaves at 203 vB each and $\mathsf{EVICT_SUBTREE}_6$ 51 at 158 vB, improving with cohort size because the interior is hashed once.

Fees and budgets. At 10 sat/vB a $D = 16$ exit costs 10,200 sat: a 1,020,000 sat balance exits for a 1% fee, 102,000 sat for 10%, and a cohort sweep keeps leaves of a few hundred thousand satoshis evictable at 50 sat/vB (Appendix D). Varops usage is 5.6 to 7.7% of the BIP-440 budget in the dense profile and 2.5 to 4.7% in the sparse, falling with depth because the budget scales with weight; `0xC0` tiers fall under the BIP-342 sigop budget with one to three checks.

9 Related Work

CoinPool [27] is the closest prior design: a shared UTXO with per-user balances and unilateral withdrawal, realised with pre-signed or covenant-committed transactions, so exit and update remain transaction trees. Channel factories [3] need all N members to sign every change and publish a tree to exit. Ark [13] batches users into rounds whose virtual outputs are leaves of a pre-signed tree; covenant Ark [14] removes the signing round with CTV or CCV templates but keeps the tree, its unrolled exit and its expiry. Timeout trees [15] and SuperScalar [38] combine covenant trees with Decker-Wattenhofer factories and laddered expiries; MATT-style trees [11,12] replace signatures by templates. Statechains [32] transfer a whole UTXO off-chain but do not divide it; `OP_VAULT` [21] is an introspection covenant that, like ours, recreates its output under a modified commitment. LayerTree differs from all of these in that the tree is data rather than transactions:

exit is one transaction of $O(D)$ bytes, the interior needs no pre-signing, the operator can evict, and the output never expires (Table 3, Appendix D).

The Merkle-sum sparse Merkle tree and its compressed proofs come from Taproot Assets [23]; here the tree is truncated and verified in script. The data-tweaked internal key follows `OP_CHECKCONTRACTVERIFY` [12]. The channel state machine is eltoo [6], with `ANYPREVOUT` [7] emulated by collated introspection and `OP_CHECKSIGFROMSTACK`. Liquid's tapscript [2] has `OP_CAT`, `OP_CHECKSIGFROMSTACK`, per-field introspection, 64-bit arithmetic and point-tweak verification, so the MID tier is close to expressible there today; Simplicity [22] would express the covenant directly. We know of no prior use of a generator-key signature with $s = e + 1$ to make a script's sighash reconstruction binding (§5.3).

10 Discussion and Limitations

The covenant guarantees conservation and the payability of every leaf under a grounded node. It does not guarantee that the operator will add you (a member who is never added has lost nothing), that grafted nodes have preimages, or that the tree was created correctly; the specification makes the members' checks mandatory and the Lean development shows they are load-bearing. The operator cannot steal, freeze provable leaves or block exits, but it can evict any leaf after its lease and, in a cohort tree, many at once for about 160 vB each, which makes griefing cheap; members who want a hard guarantee buy it with a future lease height and, for channel leaves, give up routing until it passes. Everything funnels through one UTXO, so an uncooperative mass exit is bounded to roughly a block's worth of transitions per block and exposed to pinning (§7); trees should be sized by this throughput and sharded, and this cannot be engineered away within the single-UTXO design. The design is hostage to proposals: a tapleaf version with restored string opcodes or at least `OP_CAT`, `OP_TX` in `0xC0` for the MID tier, and the EC opcodes' drafts; every selector and opcode number is a named constant in the implementation, and a Liquid prototype of the MID tier is the obvious next experiment. Future work: a P model [8] of the off-chain protocol under an adversarial scheduler, a verified interpreter connecting the Lean rules to the bytecode, and completing the MIN tier.

11 Conclusion

A shared output whose tree lives off-chain as data rather than transactions gives every member a one-transaction exit of about a kilobyte, gives the operator eviction and cohort sweeps, needs no pre-signing and never expires, using only opcodes already on the table. Its guarantees are mechanically checked theorems, its costs are measured, and its price is that all activity funnels through one UTXO, which bounds how fast a tree can be dissolved against its will.

References

1. Black, B., Rubin, J.: BIP-348: CHECKSIGFROMSTACK. https://github.com/bitcoin/bips/blob/master/bip-0348.md (2024)
2. Blockstream: Elements tapscript opcodes: Introspection, streaming SHA256, 64-bit arithmetic, ECMULSCALARVERIFY, TWEAKVERIFY. https://github.com/ElementsProject/elements/blob/master/doc/tapscript_opcodes.md (2021)
3. Burchert, C., Decker, C., Wattenhofer, R.: Scalable funding of bitcoin micropayment channel networks. In: Royal Society Open Science. vol. 5 (2018)
4. Code, R., et al.: BIP-449: OP_TWEAKADD. https://github.com/bitcoin/bips/ (2025)
5. Dahlberg, R., Pulls, T., Peeters, R.: Efficient sparse Merkle trees: Caching strategies and secure (non-)membership proofs. In: NordSec 2016. LNCS, vol. 10014, pp. 199–215. Springer (2016)
6. Decker, C., Russell, R., Osuntokun, O.: eltoo: A simple layer2 protocol for bitcoin. https://blockstream.com/eltoo.pdf (2018)
7. Decker, C., Towns, A.: BIP-118: SIGHASH_ANYPREVOUT for taproot scripts. https://github.com/bitcoin/bips/blob/master/bip-0118.mediawiki (2017)
8. Desai, A., Gupta, V., Jackson, E., Qadeer, S., Rajamani, S., Zufferey, D.: P: Safe asynchronous event-driven programming. In: PLDI 2013. pp. 321–332. ACM (2013)
9. Harding, D.A., Todd, P.: BIP-125: Opt-in full replace-by-fee signaling. https://github.com/bitcoin/bips/blob/master/bip-0125.mediawiki (2015)
10. Heilman, E., Sabouri, A.: BIP-347: OP_CAT in tapscript. https://github.com/bitcoin/bips/blob/master/bip-0347.mediawiki (2023)
11. Ingala, S.: Merkleize all the things. bitcoin-dev mailing list, https://merkle.fun (2022)
12. Ingala, S.: BIP-443: OP_CHECKCONTRACTVERIFY. https://github.com/bitcoin/bips/blob/master/bip-0443.md (2025)
13. Keceli, B.: Ark: An off-chain payment protocol. https://arkdev.info/ (2023)
14. Labs, A.: Covenant-based Ark (clArk): Rounds without presigning. https://arkdev.info/docs/learn/concepts (2024)
15. Law, J.: Scaling lightning with simple covenants. bitcoin-dev mailing list, https://github.com/JohnLaw2/ln-scaling-covenants (2023)
16. Merkle, R.C.: A digital signature based on a conventional encryption function. In: CRYPTO 1987. LNCS, vol. 293, pp. 369–378. Springer (1988)
17. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: CADE 2021. LNCS, vol. 12699, pp. 625–635. Springer (2021)
18. Myers, R., et al.: LN-Symmetry (eltoo) implementation notes. https://github.com/instagibbs/bolts/blob/eltoo_draft/XX-eltoo-transactions.md (2023)
19. Nick, J., Ruffing, T., Jin, E.: BIP-327: MuSig2 for BIP340-compatible multi-signatures. https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki (2022)
20. Nick, J., Ruffing, T., Seurin, Y.: MuSig2: Simple two-round Schnorr multi-signatures. In: CRYPTO 2021. LNCS, vol. 12825, pp. 189–221. Springer (2021)
21. O'Beirne, J., Sanders, G., Towns, A.: BIP-345: OP_VAULT. https://github.com/bitcoin/bips/blob/master/bip-0345.mediawiki (2023)
22. O'Connor, R.: Simplicity: A new language for blockchains. PLAS 2017, https://blockstream.com/simplicity.pdf (2017)
23. Osuntokun, O.: Taproot assets: Merkle sum sparse merkle trees. https://github.com/lightninglabs/taproot-assets/blob/main/docs/bip-tap-ms-smt.mediawiki (2022)

24. Osuntokun, O.: Elliptic curve operations for bitcoin script. Draft BIP, https://github.com/bitcoin/bips/pull/1945 (2025)
25. Osuntokun, O.: LayerTree: Specification, reference implementation and formal model. https://github.com/roasbeef/layertree (2026)
26. Poon, J., Dryja, T.: The bitcoin lightning network: Scalable off-chain instant payments. https://lightning.network/lightning-network-paper.pdf (2016)
27. Riard, A., Naumenko, G.: CoinPool: Efficient off-chain payment pools for Bitcoin. https://coinpool.dev/v0.1.pdf (2022)
28. Rubin, J.: BIP-119: CHECKTEMPLATEVERIFY. https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki (2020)
29. Russell, R.: OP_TX: Transaction introspection for tapscript. Draft BIP, https://github.com/rustyrussell/bips/pull/1 (2025)
30. Russell, R., Towns, A.: BIP-440: Varops budget for script runtime constraint. https://github.com/bitcoin/bips/blob/master/bip-0440.md (2025)
31. Russell, R., Towns, A.: BIP-441: Restoration of disabled script opcodes (tapleaf 0xC2). https://github.com/bitcoin/bips/blob/master/bip-0441.md (2025)
32. Somsen, R.: Statechains: Off-chain transfer of UTXO ownership. In: Scaling Bitcoin 2018 (2018)
33. Wuille, P., Daftuar, S., Zhao, G.: Cluster mempool. Delving Bitcoin, https://delvingbitcoin.org/t/an-overview-of-the-cluster-mempool-proposal/393 (2023)
34. Wuille, P., Nick, J., Ruffing, T.: BIP-340: Schnorr signatures for secp256k1. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki (2020)
35. Wuille, P., Nick, J., Towns, A.: BIP-341: Taproot: SegWit version 1 spending rules. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki (2020)
36. Wuille, P., Nick, J., Towns, A.: BIP-342: Validation of taproot scripts. https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki (2020)
37. Zhao, G.: BIP-431: Topology restrictions for pinning. https://github.com/bitcoin/bips/blob/master/bip-0431.mediawiki (2024)
38. ZmnSCPxj: SuperScalar: Laddered timeout-tree-structured Decker-Wattenhofer factories. Delving Bitcoin, https://delvingbitcoin.org/t/superscalar-laddered-timeout-tree-structured-decker-wattenhofer-factories/1143 (2024)

A The Lean Formalisation

This appendix describes the mechanised model behind §7 in enough detail that a reader can judge what has been proved, under what assumptions, and what has not. The development is about 1,300 lines of Lean 4 [17] in seven modules and depends on nothing but the core library: no Mathlib, no external tactics. It builds in seconds and prints an axiom audit for every headline theorem; each depends only on propext, Quot.sound and, in one case, Classical.choice, and none on sorryAx. The code is in the artefact [25].

A.1 Design of the Model

The model states the *rules* of §4.3 as relations over the objects the scripts manipulate: nodes, paths, amounts. It does not model tapscript bytecode, signatures,

keys or lock times. Authorisation is carried as an opaque proposition, and the input and output amounts the scripts read with `OP_TX` are parameters. Three choices keep the proofs short.

Depth as a type index. A tree of depth d is `Tree d`, a perfect binary tree whose tips are `Option Leaf`. Positions are `Pos d h`, paths from depth d down to a subtree of height h, so that `Pos d 0` addresses a leaf and `Pos d h` addresses the slot a JOIN_h fills. Indexing by both endpoints avoids all arithmetic on depths: the recursions on `getSub`, `setSub` and `pathSub` pattern-match on the position and the tree simultaneously, and the one impossible case (a `cons` step below height 0) is dismissed by `Pos.height_le`.

```
inductive Tree : Nat → Type
  | tip : Option Leaf → Tree 0
  | bin {d : Nat} : Tree d → Tree d → Tree (d + 1)

inductive Pos : Nat → Nat → Type
  | nil {h : Nat} : Pos h h
  | cons {d h : Nat} : Bool → Pos d h → Pos (d + 1) h

def parent (a b : Node H) : Node H :=
  ⟨M.hNode a.hash a.sum b.hash b.sum, a.sum + b.sum⟩

def combine (n : Node H) (x : Bool × Node H) : Node H :=
  if x.1 then parent x.2 n else parent n x.2

def fold (n : Node H) : List (Bool × Node H) → Node H
  | [] => n
  | x :: rest => combine (fold n rest) x
```

Root-first paths. The model orders a sibling path from the root down, so `fold` is a structural recursion on the list and the soundness proof is a plain induction. The witness of §5 supplies the same pairs leaf-first, because the script consumes the deepest level first; nothing in the theorems depends on the order.

Hashes as parameters. Everything is parametric in a class `HashModel` whose fields are the two hash functions, the zero digest, and the assumptions the argument uses:

```
class HashModel (Leaf : Type) (H : outParam Type) where
  hLeaf : Leaf → H
  hNode : H → Nat → H → Nat → H
  empty0 : H
  hLeaf_inj : ∀ {l₁ l₂}, hLeaf l₁ = hLeaf l₂ → l₁ = l₂
  hNode_inj : ∀ {a₁ b₁ a₂ b₂ sa₁ sb₁ sa₂ sb₂},
    hNode a₁ sa₁ b₁ sb₁ = hNode a₂ sa₂ b₂ sb₂ →
      a₁ = a₂ ∧ sa₁ = sa₂ ∧ b₁ = b₂ ∧ sb₁ = sb₂
  hNode_ne_hLeaf : ∀ a sa b sb l, hNode a sa b sb ≠ hLeaf l
  hNode_ne_empty : ∀ a sa b sb, hNode a sa b sb ≠ empty0
  hLeaf_ne_empty : ∀ l, hLeaf l ≠ empty0
```

Injectivity is collision resistance idealised; the three inequalities are the tag separation that distinct BIP-340 tags provide. The encodings module discharges the serialisation half of each injectivity field (Lemma 1): given SHA-256 collision resistance, `serializeLeaf_inj` and `serializeNode_inj` are what remains. Nothing quantifies the adversary's advantage; a proof under injectivity transfers to any adversary that cannot exhibit a collision.

A.2 Theorem Statements

The statements below are verbatim from the development. t.root is the root node of a tree, t.getSub p the subtree at position p, t.setSub p u the tree with that subtree replaced by u, t.pathSub p the honest sibling path, and t.get, t.set, t.path their leaf-level specialisations.

```
theorem fold_pathSub (t : Tree d) (p : Pos d h) :
    fold (t.getSub p).root (t.pathSub p) = t.root

theorem fold_setSub (t : Tree d) (p : Pos d h) (s : Tree h) :
    fold s.root (t.pathSub p) = (t.setSub p s).root

theorem root_inj (t₁ t₂ : Tree d) : t₁.root = t₂.root → t₁ = t₂

theorem fold_sound (t : Tree d) (m : Node H) (π : List (Bool × Node H)) :
    fold m π = t.root →
    ∃ (h : Nat) (p : Pos d h), m = (t.getSub p).root ∧ π = t.pathSub p
```

fold_sound is Theorem 1. Its proof is the only place the hash axioms are used: a non-empty path claims an internal node, so the tree is not a tip by tag separation, and injectivity of hNode equates both children's hashes and sums with the claimed ones. Note that the conclusion equates *nodes*, so the sum the prover presented is the true sum of the subtree.

A Step is a path, a height, an old node and a new node; it is valid for roots r, r' when the old node folds to r and the new node to r', and well formed for depth d when the path has $d - h$ entries. Steps chains steps, each against the root the previous one produced.

```
theorem Step.sound (t : Tree d) (s : Step H) (r' : Node H)
    (hwf : s.WF d) (hv : s.valid t.root r') :
    ∃ p : Pos d s.height, s.old = (t.getSub p).root ∧
      ∀ (u : Tree s.height), u.root = s.new → r' = (t.setSub p u).root

theorem Steps.sum (h : Steps r ss r') :
    r'.sum + oldSum ss = r.sum + newSum ss

theorem Steps.sound (t : Tree d) (h : Steps t.root ss r')
    (hwf : ∀ s ∈ ss, s.WF d) (hg : ∀ s ∈ ss, s.Grounded) :
    ∃ t' : Tree d, t'.root = r'
```

Steps.sum is Theorem 2 and needs no hash assumption. Steps.sound is Theorem 4; s.Grounded says the new node is the root of some tree of the step's height, which leaf-level steps satisfy by construction (grounded_of_leaf) and which a JOIN graft must be given.

The transitions are structures bundling the steps with the output rule and an opaque authorisation. Their theorems:

```
theorem payable (t : Tree d) (p : Pos d 0) (l : Leaf) (hl : t.get p = some l) :
    (leafStep (t.path p) (some l) none).valid t.root (t.set p none).root ∧
    (t.set p none).root.sum + l.amount = t.root.sum ∧
    ∀ q, q ≠ p → (t.set p none).get q = t.get q

theorem ExitK.no_inflation (h : ExitK exact r blocks r' out0) :
    out0 + payoutSum blocks ≤ r.sum

theorem ExitK.exact_conservation (h : ExitK true r blocks r' out0) :
    out0 + payoutSum blocks = r.sum

theorem join_sound (t : Tree d) (hwf : path.length + h = d)
    (hj : Join auth h t.root path new r' out0) :
```

```
    ∃ p : Pos d h, t.getSub p = Tree.empty h ∧
      out0 = t.root.sum + new.sum ∧
      ∀ u : Tree h, u.root = new → r' = (t.setSub p u).root

theorem claim_sound (t : Tree d) (hwf₁ : srcPath.length = d)
    (hwf₂ : claimPath.length = d)
    (hc : Claim auth t.root srcPath claimPath src claim r' out0) :
    out0 = t.root.sum ∧
    ∃ (p₁ p₂ : Pos d 0), t.get p₁ = some src ∧
      (t.set p₁ (claimedSource src claim.amount)).get p₂ = none ∧
      r' = ((t.set p₁ (claimedSource src claim.amount))
              .set p₂ (some claim)).root
```

payable is Theorem 3. join_sound uses root_inj (via eq_empty_of_root): since JOIN_h hard-codes E_h as the old node, the slot it fills is provably the empty tree, and the theorem gives the successor root for every tree whose root is the grafted node. claim_sound exposes the drained-source case: the claim position must be empty in the tree *after* the source step, which is why a source reduced to zero may host the claim.

A.3 Assumption Ledger

Every hypothesis outside Lean's logic, what it stands for in the deployment, and which theorems consume it.

hypothesis	stands for	used by
hLeaf_inj	SHA-256 collision resistance + serialization	leafNode_inj, root_inj
hNode_inj	SHA-256 collision resistance + serialization	root_inj, fold_sound
hNode_ne_hLeaf, hNode_ne_empty, hLeaf_ne_empty	distinct tags (random-oracle heuristic)	leafNode_inj, fold_sound
treeSpk_inj	DL on secp256k1 + CR of H_{state}, TapTweak	witness_root_binds
real-tree invariant	creation check + grounded grafts (off-chain)	every theorem stated for a Tree
(none)		Steps.sum, ExitK.no_inflation, encodings

The conservation results use no hypothesis at all, which is the formal content of "the covenant conserves sums whatever it is shown". The collision-resistance assumption on the TapSighash and challenge hashes that the MIN tier's constructive introspection adds (§5.3) is outside the model, since the model does not include signature checking.

A.4 Modules and Effort

module	lines	contents
Hash.lean	60	the HashModel class
Leaf.lean	100	Leaf, Leaf.WF, Node, leafNode, leafNode_inj
Tree.lean	330	trees, positions, paths, fold, all tree theorems
Transitions.lean	460	Step, Steps, the transitions and their theorems
Binding.lean	60	BindingModel and the two binding theorems
Encoding.lean	230	encodings, serialisation injectivity, SEQ_FINAL
Axioms.lean	35	the axiom audit

The only proofs that took real effort were fold_sound, where the two-index positions and the domain-separation case for a tip had to be found, and Steps.sound, where the height of the grounding tree must be reconciled with the height the soundness lemma returns, which is why Step carries its height explicitly. A verified interpreter for the tapscript subset would need a model of the stack machine, of BIP-441 numerics, of OP_TX's serialisations and of the BIP-341 message, and a proof that each generated script's execution relation equals the corresponding Step relation; we estimate it at several thousand lines and regard it as the natural sequel.

A.5 Binding to the Output

Binding.lean records the one cryptographic step outside the hash model. A structure BindingModel carries an abstract treeSpk : H → TapRoot → Spk with the hypothesis that it is injective in both arguments, and two one-line theorems conclude that a witness (r', m') passing the scriptPubKey check equals the committed (r, m), and that the successor output commits to the computed root under the same m. In the deployment the map is $(r, m) \mapsto x\big(P_r + H_{\mathsf{TapTweak}}(x(P_r) \,\|\, m) \cdot G\big)$ with $P_r = \mathsf{lift_x}(x(\mathsf{NUMS} + H_{\mathsf{state}}(r)G))$, and the two x-projections identify a point with its negation. A collision $(r, m) \neq (r', m')$ yields one of: $x(P_r) = x(P_{r'})$ with $r \neq r'$, which is a collision in H_{state} or the relation $\mathsf{NUMS} + tG = -(\mathsf{NUMS} + t'G)$, solvable for the discrete logarithm of NUMS; $x(P_r) = x(P_{r'})$ with $m \neq m'$, a collision in H_{TapTweak}; or distinct internal keys with $Q = \pm Q'$, again a linear relation in NUMS and G solvable for the discrete logarithm of NUMS. Making the hypothesis explicit is the point: it is what a future script change must not break.

A.6 Encodings

Encoding.lean proves the byte-level facts the design leans on, over natural-number bytes so that every proof stays within linear arithmetic: fixed-width little-endian encodings are injective on their range; minimalLE (n + 2^(8k)) = leBytes k n ++ [1] for $n < 2^{8k}$, which with $k = 8$ is enc_{64} and justifies producing it with one addition; the 116-byte leaf and the 82-byte node preimage are injective; and the SEQ_FINAL facts of Lemma 1, computed by decide on a bounded-recursion model of CScriptNum.

A.7 What Is Not Proved

The tapscript bytecode is outside the model. That the scripts of §5 implement the relations above is established by execution: every script of every tier runs in the simulator against a reference tree with positive and negative cases, and the Go generators are compared byte for byte with the Python output. A verified interpreter for the tapscript subset (BIP-441 numerics, OP_TX, OP_CHECKSIGFROMSTACK) would close this gap and is a project of a different size. Signatures, lock times

and the transaction structure beyond amounts are authorisation-side facts and are deliberately opaque. Concurrency, mempool policy, reorganisations and the off-chain protocol are not theorems about hash trees and belong to a model checker; a design for a P [8] model is in the artefact.

B Script Listings

Figure 3 states the step every transition is built from; the listings below are taken from the reference generators and are what the size figures of §8 measure. Stack comments show the main stack left to right with the top on the right, and the alt stack after a bar.

$\mathsf{Step}(\pi, n_{\text{old}}, n_{\text{new}}, (r, S))$

1 : $(r_0, S_0) \leftarrow \mathsf{fold}(n_{\text{old}}, \pi)$ // $D - h$ unrolled levels
2 : **assert** $r_0 = r \wedge S_0 = S$ // CHECK_OLD: S is input 0's amount
3 : $(r', S') \leftarrow \mathsf{fold}(n_{\text{new}}, \pi)$ // same siblings, same sums
4 : **return** (r', S')

$\mathsf{Final}((r', S'), m)$

1 : **if** $(r', S') = E_D$ **then return** // terminal: output 0 free
2 : $(a_0, \mathsf{spk}_0) \leftarrow$ OP_TX(output 0)
3 : **assert** $\mathsf{spk}_0 = \mathsf{TreeSPK}(r', m) \wedge a_0 = S'$

Fig. 3. The Merkle-sum step shared by all transitions, and the output rule. A transition is a sequence of steps, each proving its old node against the root the previous step produced (CHECK_PREV), followed by Final. Authorisation and payout checks are specific to each transition.

One level of the fold (FULL tier, 88 bytes). LEVEL advances both accumulators, the old node (oh, os) and the new node (nh, ns), by one level using the same direction bit and sibling $sc = h \,\|\, \mathsf{enc}_{64}(s)$ from the witness. T is the 64-byte tag prefix of H_{node}; PAD8 is $\langle 2^{64} \rangle$ OP_ADD, which yields enc_{64} of the sum by Lemma 1.

```
OP_6 OP_ROLL OP_6 OP_ROLL            # T oh os nh ns dir sc
OP_SIZE 41 OP_EQUALVERIFY            #   sc is exactly hash || enc64(sum)
OP_DUP 32 OP_8 OP_SUBSTR             # ... dir sc ss8   (sibling sum, raw le64)
OP_DUP OP_TOALTSTACK                 #                          | ss8
OP_3 OP_ROLL OP_TUCK OP_ADD          # T oh os nh dir sc ns ns'
OP_TOALTSTACK PAD8                   # T oh os nh dir sc ns9    | ss8 ns'
OP_3 OP_ROLL OP_SWAP OP_CAT          # T oh os dir sc nc        (nc = nh || enc64(ns))
OP_OVER OP_SWAP OP_3 OP_PICK         # T oh os dir sc sc nc dir
OP_NOTIF OP_SWAP OP_ENDIF OP_CAT     # T oh os dir sc pre_n     (dir ? sc||nc : nc||sc)
OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 # T oh os dir sc nh'
OP_TOALTSTACK                        #                          | ss8 ns' nh'
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
                                     # T oh os dir sc nh' ns' ss8
OP_5 OP_ROLL OP_TUCK OP_ADD          # T oh dir sc nh' ns' os os'
OP_TOALTSTACK PAD8                   # T oh dir sc nh' ns' os9  | os'
```

```
OP_5 OP_ROLL OP_SWAP OP_CAT           # T dir sc nh' ns' oc      (oc = oh || enc64(os))
OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL     # T nh' ns' sc oc dir
OP_NOTIF OP_SWAP OP_ENDIF OP_CAT      # T nh' ns' pre_o
OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256 # T nh' ns' oh'
OP_FROMALTSTACK OP_2SWAP              # T oh' os' nh' ns'
```

Checking the old root and writing the new one. After D levels, CHECK_OLD compares the old accumulator with the committed state: the sum against input 0's amount and the hash against the witness root r parked on the alt stack (itself bound to the input scriptPubKey by the prologue). FINAL constrains output 0 unless the new root is the empty tree.

```
# CHECK_OLD:  [T oh os nh ns] | ... r  ->  [T nh ns]
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

# FINAL:  [T nh ns] | m  ->  [1]
OP_ROT OP_DROP                                  # nh ns
OP_OVER <E_D.hash> OP_EQUAL
OP_IF   OP_2DROP OP_FROMALTSTACK OP_DROP OP_1   # terminal: output 0 free
OP_ELSE OP_SWAP OP_FROMALTSTACK TREE_SPK        # ns spk'
        OP_0 <SEL_OUT_SINGLE> OP_TX             # ns spk' amt0 spk0
        OP_ROT OP_EQUALVERIFY OP_NUMEQUAL       # 1
OP_ENDIF
```

The MID gadget. Under `0xC0` a computed 4-byte number x is widened to 8 bytes by two witness elements z, w: the script checks $\mathsf{size}(x) + \mathsf{size}(z) = 4$ and $z \,\|\, w = \texttt{00000000}$ and uses $x \,\|\, z \,\|\, w$ as $\mathsf{le}_{64}(x)$. Two such gadgets per level and the per-field size checks account for almost all of MID's premium over FULL.

The MIN message. The MIN exit assembles the BIP-341 signature message for input 0 with hash type `SIGHASH_DEFAULT` and checks it as described in §5.3. Fields marked *derived* are computed by the script from values it has verified; fields marked *raw* are witness elements the script does not constrain but must be presented consistently, since the whole message is hashed and compared with the real one through `OP_CHECKSIG`.

field	source	note
epoch, hash type	constant 00 00	SIGHASH_DEFAULT
version, lock time	raw	free
sha_prevouts	raw	free
sha_amounts	derived: $\mathsf{SHA256}(\mathsf{le}_{64}(S) \,\|\, amt_1)$	S verified by CHECK_OLD; amt_1 raw; two inputs exactly
sha_scriptpubkeys	derived: $\mathsf{SHA256}(22 \,\|\, \mathsf{TreeSPK}(r, m) \,\|\, spk_1)$	TreeSPK recomputed; spk_1 raw
sha_sequences	raw	the field the spender grinds
sha_outputs	derived: $\mathsf{SHA256}(out_0 \,\|\, out_1 \,\|\, extra)$	$out_0 = \mathsf{le}_{64}(S') \,\|\, 22 \,\|\, \mathsf{TreeSPK}(r', m)$, out_1 the payout; $extra$ raw
spend type, input index	constant 02 00000000	script path, no annex, input 0
tapleaf hash	raw	the leaf being executed
key version, codesep	constant 00 ffffffff	

The message is hashed with the TapSighash tag and the challenge e is computed from $G_x \,\|\, G_x \,\|\, \mathsf{msg}$ under the challenge tag. The script verifies $e_{[0,31)} \,\|\, \texttt{00} = e$ against the witness-supplied $e_{[0,31)}$ and finally executes

$$\langle G_x \,\|\, e_{[0,31)} \,\|\, \texttt{01} \rangle \;\; \langle G_x \rangle \;\; \texttt{OP_CHECKSIG}.$$

C Channel Leaves: Scripts and Costs

Selectors and context. Two `OP_TX` selectors are used, both with the collate flag so that one call leaves one blob: SEL_CHAN_UPD selects version, number of inputs, the current input's sequence, number of outputs, all output amounts and scripts, and lock time; SEL_CHAN_SET adds the current input's amount and scriptPubKey. The per-channel context is

$$\mathsf{chan_id} = H_{\mathsf{chanid}}(m \,\|\, \mathsf{owner_key} \,\|\, \mathsf{le}_{32}(\mathsf{nonce}) \,\|\, x(P_{\mathsf{chan}}) \,\|\, \mathsf{le}_{64}(C) \,\|\, \mathsf{le}_{32}(\mathsf{epoch}) \,\|\, \mathsf{le}_{32}(\mathsf{csv_delay})),$$

with $\mathsf{ctx}_{\mathsf{upd}}$ and $\mathsf{ctx}_{\mathsf{set}}$ two tagged hashes of it, so that an update signature never verifies where a settlement signature is required. The signed messages are $\mathsf{SHA256}(\mathsf{ctx} \,\|\, \texttt{OP_TX}(\mathsf{SEL}))$.

Update output scripts. The update transaction for state n pays its whole input value to an output U_n with internal key P_{chan} (a MuSig2 [20,19] key of the two parties) and the two-leaf taptree of Figure 4. A later update binds to U_n through REBIND_n, whose lock-time check admits only states above n; the settlement of state n binds through SETTLE_n after csv_delay blocks, and only to U_n because SEL_CHAN_SET commits the spent script, which embeds n.

REBIND_n (tapleaf 0xC2, 81 B)
$\langle S_0 + n + 1 \rangle$ `OP_CHECKLOCKTIMEVERIFY OP_DROP`
$\langle$SEL_CHAN_UPD$\rangle$ `OP_TX` // collated blob
$\langle \mathsf{ctx}_{\mathsf{upd}} \rangle$ `OP_SWAP OP_CAT OP_SHA256`
$\langle P_{\mathsf{chan}} \rangle$ `OP_CHECKSIGFROMSTACK`

SETTLE_n (tapleaf 0xC2, 80 B)
$\langle$csv_delay$\rangle$ `OP_CHECKSEQUENCEVERIFY OP_DROP`
$\langle$SEL_CHAN_SET$\rangle$ `OP_TX` // + input amount, spk
$\langle \mathsf{ctx}_{\mathsf{set}} \rangle$ `OP_SWAP OP_CAT OP_SHA256`
$\langle P_{\mathsf{chan}} \rangle$ `OP_CHECKSIGFROMSTACK`

Fig. 4. The two leaves of a channel leaf's update output for state n.

Transactions. The update transaction is version 3, lock time $S_0 + n$, one input with sequence `0xFFFFFFFD`, one output of the full capacity C to U_n and a zero-value pay-to-anchor output; it pays no fee and is valid whether bound to the funding output or to any earlier update output, both of value C. The settlement is version 3 with input sequence csv_delay, outputs for the two balances and each in-flight HTLC, and an anchor; $b_U + b_V + \sum \text{htlc} = C$. Fees for both come from a child spending the anchor, so both parties need an on-chain fee reserve to close unilaterally. The signing order is settlement before update for each state, as in eltoo.

Force-close cost. At $D = 16$ dense, csv_delay $= 144$ and no HTLCs in flight:

step	vB earliest
EXIT_1 by owner or evictor, with fee input and change	$\approx 1{,}020\ t$
update_n bound to the funding output	$153\ t+1$
anchor child for update_n	$152\ t+1$
settlement_n (two payouts, anchor)	$204\ t+1+\mathsf{csv}$
anchor child for settlement_n	$152\ t+1+\mathsf{csv}$
total	$\approx 1{,}680 \approx 146$ blocks
contested rebind to an earlier update output	161 per contest
settlement with h HTLC outputs	$204+43h$

The lifetime on-chain footprint of a channel leaf that is force-closed is therefore about 1,680 vB, against roughly 280 vB for a Lightning channel's funding transaction plus force close; cooperative closes, splices and lease renewals of a channel leaf cost one tree transition or nothing. The update transaction cannot be packaged with the exit because a version-3 transaction may not spend an unconfirmed version-2 parent, so G_{exit} in §6 is measured to confirmation.

D Vouchers and Fee Tables

D.1 Vouchers (CLAIM)

A voucher is a BIP-340 signature by the source leaf's owner over

$$H_{\mathsf{voucher}}(m \,\|\, \ell_{\text{src}} \,\|\, \ell_{\text{claim}}),$$

verified with `OP_CHECKSIGFROMSTACK`. CLAIM is two steps: the source leaf's amount is reduced by the claim amount and its seq advanced by one (or the position emptied if drained), then the claim leaf is placed at an empty position. The total is unchanged. Replay is impossible because the first step changes the source leaf, so the signed message never again matches a leaf in the tree; seq prevents a balance that later returns to an earlier value from re-enabling an old voucher. Because a voucher lets an owner move value with a lone signature, leaves whose balance is not the owner's alone set seq to the tier's SEQ_FINAL (§6), whose successor does not fit in four bytes, so the CLAIM script rejects them. Vouchers are the mechanism for operator-to-member push payments without the operator's later participation, and with a payment hash added to the message they become HTLCs between the credit instrument and channel-leaf HTLCs.

D.2 Comparison Table

D.3 Fee Tables

Fees for the measured sizes at fee rates from 1 to 250 sat/vB, FULL tier. The exogenous-fee skeleton is 946 WU for one payout plus 172 WU per additional payout; vB = $\lceil \text{WU}/4 \rceil$.

Table 3. Comparison with shared-output constructions.

	LayerTree	CoinPool	Ark (covenant)	Timeout tree	Factory
Unilateral exit	1 tx, $O(D)$ B	$O(\log N)$ txs	$O(\log N)$ txs	$O(\log N)$ txs	$O(\log N)$ txs
Evict one leaf	1 tx after lease	no	tree expires	tree expires	no
Interactive signing	none	all	none	none or all	all
Shared output expiry	none	none	per round	per tree	none
Batch add	1 tx, any subtree	re-sign all	per round	per tree	re-sign all
Opcodes	GSR, `OP_TX`, CSFS, EC	CTV or APO	CTV / CCV	CTV	none

case	vB	1	10	25	50	250 sat/vB
dense EXIT_1, $D = 8$	762	762	7,620	19,050	38,100	190,500
dense EXIT_1, $D = 16$	1,020	1,020	10,200	25,500	51,000	255,000
dense EXIT_1, $D = 20$	1,149	1,149	11,490	28,725	57,450	287,250
sparse EXIT_1, $D = 32$, $N = 6$	1,637	1,637	16,370	40,925	81,850	409,250
MID dense EXIT_1, $D = 16$	1,338	1,338	13,380	33,450	66,900	334,500
MIN dense EXIT_1, $D = 16$	1,456	1,456	14,560	36,400	72,800	364,000
second exit in EXIT_2, marginal	665	665	6,650	16,625	33,250	166,250
$\mathsf{EVICT_SUBTREE}_4$, per leaf (13)	203	203	2,029	5,073	10,146	50,731
$\mathsf{EVICT_SUBTREE}_6$, per leaf (51)	158	158	1,580	3,950	7,899	39,495
plain P2TR to P2TR	111	111	1,110	2,775	5,550	27,750

Break-even balances for the dense $D = 16$ exit, i.e. the leaf amount at which the fee is 1%, 5% or 10% of the balance:

sat/vB	fee (sat)	balance at 1%	at 5%	at 10%
2	2,040	204,000	40,800	20,400
10	10,200	1,020,000	204,000	102,000
50	51,000	5,100,000	1,020,000	510,000
250	255,000	25,500,000	5,100,000	2,550,000

Operators should size cohorts and minimum leaf amounts so that a sweep stays below about 10% of the smallest leaf at the fee-rate tail they intend to survive; the FULL-tier fee-from-payout option saves a constant 99 vB and is unavailable to channel leaves.

E Complete Script Listings

This appendix reproduces every script the reference generators emit, for the smallest depths that exercise every construct: dense $D = 4$ and sparse $D = 8$. Tapscript has no subroutines, so a compiled script is one flat opcode stream; the generators, however, build every script from a small set of helpers, and the listings below are those streams *collapsed back* into named macro calls, one per line, with the remaining raw opcodes wrapped between them. Each tier's macros are defined once in §E.1 to §E.8, themselves collapsed against the other macros. The collapse is mechanical (`tools/gen_structured.py`) and verified: expanding a structured listing reproduces the compiled bytes exactly, so these listings and the flat ones in `spec/scripts/` describe the same scripts, and the byte count in

each heading is the compiled size. A run of n identical calls is written `NAME` xn. Angle brackets denote pushes of named constants: `<TAG_NODE>` is the 64-byte tagged-hash prefix, `<EMPTY_k>` the hash of E_k, `<ENC0>` the constant 2^{64} whose addition implements PAD8, `<NUMS>` the BIP-341 nothing-up-my-sleeve point, and `<SEL_*>` the `OP_TX` selectors of §4.2. Bare integers are minimal number pushes. Sizes at $D = 16$ and $D = 32$ are in Table 2; each further level adds one LEVEL (88 bytes dense, 135 sparse, 156 MID).

E.1 FULL Tier Macros

```
# Macro definitions, FULL tier. Each expansion is itself collapsed
# against the other macros; a macro never contains itself.

## TAGGED(LEAF)
<TAG_LEAF> OP_SWAP OP_CAT OP_SHA256

## TAGGED(NODE)
<TAG_NODE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(STATE)
<TAG_STATE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(VOUCHER)
<TAG_VOUCHER> OP_SWAP OP_CAT OP_SHA256

## TAGGED(POS)
<TAG_POS> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPTWEAK)
<TAG_TAPTWEAK> OP_SWAP OP_CAT OP_SHA256

## PAD8   ([n] -> [enc64(n)]: minimal encoding of n + 2^64 is le64(n)||01)
<ENC0> OP_ADD

## LE64   ([n] -> [le64(n)])
PAD8
OP_8 OP_LEFT

## PAD4   ([n] -> [le32(n)])
<ZERO4> OP_CAT OP_4 OP_LEFT

## TREE_SPK   ([r m] -> [TREE_SPK(r, m)] with EC ops)
OP_SWAP
TAGGED(STATE)
OP_0 OP_SWAP OP_EC_POINT_MUL <NUMS> OP_EC_POINT_ADD OP_EC_POINT_X_COORD
  OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_0 OP_SWAP OP_EC_POINT_MUL OP_SWAP OP_2 OP_SWAP OP_CAT OP_EC_POINT_ADD
  OP_EC_POINT_X_COORD <0x5120> OP_SWAP OP_CAT

## TREE_SPK_MIN   ([r m] -> [TREE_SPK(r, m)] with OP_TWEAKADD)
OP_SWAP
TAGGED(STATE)
<NUMS_X> OP_TWEAKADD OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_SWAP OP_TWEAKADD <0x5120> OP_SWAP OP_CAT

## LEAF_ACC   ([leaf] -> [H_leaf(leaf) amount])
OP_DUP OP_8 OP_LEFT OP_SWAP
TAGGED(LEAF)
OP_SWAP

## LEAF_POS   ([leaf] -> [leaf H_pos(owner_key||nonce)])
```

```
OP_DUP 40 32 OP_SUBSTR OP_OVER 108 OP_4 OP_SUBSTR OP_CAT
TAGGED(POS)

## PROLOGUE   (input is #0; (r, m) match the input scriptPubKey; alt <- m r)
<SEL_CURR_INPUT> OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK

## LEVEL   (dense level: dir sc T oh os nh ns -> T oh' os' nh' ns')
OP_6 OP_ROLL OP_6 OP_ROLL OP_SIZE 41 OP_EQUALVERIFY OP_DUP 32 OP_8 OP_SUBSTR
  OP_DUP OP_TOALTSTACK OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
PAD8
OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD
  OP_TOALTSTACK
PAD8
OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK OP_2SWAP

## LEVEL_MIN   (dense level without splice opcodes: sibling as two elements)
OP_7 OP_ROLL OP_7 OP_ROLL OP_7 OP_ROLL OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER
  OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_DUP OP_TOALTSTACK OP_SWAP OP_OVER
PAD8
OP_CAT OP_SWAP OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
PAD8
OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD
  OP_TOALTSTACK
PAD8
OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK OP_2SWAP

## SPARSE_SETUP   ([pos_hash] -> [] ; alt <- pos E_0)
OP_TOALTSTACK <EMPTY_0> OP_TOALTSTACK

## SPARSE_TEARDOWN   (drop pos and E_k from the alt stack)
OP_FROMALTSTACK OP_DROP OP_FROMALTSTACK OP_DROP

## CHECK_OLD   (os == input 0 amount, oh == r (from alt))
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY OP_FROMALTSTACK
  OP_EQUALVERIFY

## CHECK_OLD_S   (CHECK_OLD with the sparse teardown)
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV   (os == ns_prev, oh == nh_prev (from alt); chained steps)
OP_2SWAP OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV_S   (CHECK_PREV with the sparse teardown)
OP_2SWAP
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## LEAF_STEP   (cohort sweep: one leaf or empty -> its node, paying the next output)
OP_SIZE OP_0NOTEQUAL OP_IF OP_SIZE 116 OP_EQUALVERIFY OP_DUP 72 32 OP_SUBSTR
  <OPERATOR_KEY> OP_EQUALVERIFY OP_DUP 104 OP_4 OP_SUBSTR
  OP_CHECKLOCKTIMEVERIFY OP_DROP OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK
  <SEL_OUT_SINGLE> OP_TX <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT
  OP_EQUALVERIFY OP_OVER OP_8 OP_LEFT OP_NUMEQUALVERIFY
LEAF_ACC
OP_ELSE OP_DROP <EMPTY_0> OP_0 OP_ENDIF
```

```
## LEVEL_S(k)   [shown for k = 0; <EMPTY_k> and the bit index vary with k]   (sparse level
     k: sc T oh os nh ns | pos E_k -> T oh' os' nh' ns' | pos E_k+1)
OP_5 OP_ROLL OP_FROMALTSTACK OP_SWAP OP_SIZE OP_ONOTEQUAL OP_NOTIF OP_DROP
  OP_DUP <ENC0> OP_CAT OP_ENDIF OP_SWAP <ENC0> OP_CAT OP_DUP OP_CAT OP_6
  OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_DUP OP_TOALTSTACK
  OP_SWAP OP_TOALTSTACK OP_0 OP_DOWNSHIFT OP_1 OP_AND OP_ONOTEQUAL OP_SWAP
  OP_SIZE 41 OP_EQUALVERIFY OP_DUP 32 OP_8 OP_SUBSTR OP_DUP OP_TOALTSTACK
  OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
PAD8
OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD
  OP_TOALTSTACK
PAD8
OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK OP_2SWAP

## FINAL   [shown for D = 4; <EMPTY_D> is the empty root of the tree's depth]   (output 0 is
     (ns, TREE_SPK(nh, m)) unless nh == E_D)
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_FROMALTSTACK
  OP_DROP OP_1 OP_ELSE OP_SWAP OP_FROMALTSTACK
TREE_SPK
OP_0 <SEL_OUT_SINGLE> OP_TX OP_ROT OP_EQUALVERIFY OP_NUMEQUAL OP_ENDIF

## HASH_STEP(t)   [shown for t = 3; the OP_PICK depth is t-3]   (cohort sweep: two child
     nodes -> parent node)
OP_2 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK
PAD8
OP_CAT OP_SWAP
PAD8
OP_ROT OP_SWAP OP_CAT OP_SWAP OP_CAT OP_0 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK
```

E.2 FULL Tier, Dense Profile ($D = 4$)

```
# EXIT_1 (dense, D=4): 1045 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_1 <SEL_OUT_SINGLE> OP_TX
  <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8
  OP_LEFT OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_OLD
FINAL

# EXIT_2 (dense, D=4): 1635 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_1 <SEL_OUT_SINGLE> OP_TX
  <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8
  OP_LEFT OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_DUP OP_TOALTSTACK OP_IF
```

```
  OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY OP_DROP OP_OVER 72 32
  OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF OP_CHECKSIGVERIFY
  OP_SIZE 116 OP_EQUALVERIFY OP_2 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_3
  OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8 OP_LEFT
  OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_PREV
FINAL

# JOIN_0 (dense, D=4): 961 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_OLD
FINAL

# JOIN_2 (dense, D=4): 785 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_2> OP_0 OP_2SWAP
LEVEL  x2
CHECK_OLD
FINAL

# JOINK_2 (dense, D=4): 1433 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_SIZE OP_8 OP_EQUALVERIFY
  OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT
  <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL

# UPDATE (dense, D=4): 1081 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116
  OP_EQUALVERIFY <TAG_NODE> OP_ROT OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL  x4
CHECK_OLD
FINAL

# UPDATE_X (exit-key consent, dense, D=4): 1088 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_4 OP_PICK OP_8 32 OP_SUBSTR OP_CHECKSIGVERIFY <OPERATOR_KEY>
  OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGVERIFY OP_SIZE
```

```
  116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116 OP_EQUALVERIFY <TAG_NODE> OP_ROT
  OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL  x4
CHECK_OLD
FINAL

# CLAIM (dense, D=4): 1690 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

<SEL_CURR_INPUT> OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_DUP OP_TOALTSTACK OP_SWAP OP_TOALTSTACK
  OP_3 OP_PICK OP_3 OP_PICK OP_CAT OP_CAT
TAGGED(VOUCHER)
OP_3 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGFROMSTACK OP_VERIFY OP_SIZE 116
  OP_EQUALVERIFY OP_SWAP OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_OVER OP_8
  OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB
LE64
OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY
PAD4
OP_CAT OP_FROMALTSTACK OP_ROT OP_TOALTSTACK OP_TOALTSTACK <TAG_NODE> OP_ROT
  OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
OP_DUP OP_NOT OP_IF OP_SWAP OP_DROP <EMPTY_0> OP_SWAP OP_ENDIF
LEVEL  x4
CHECK_OLD
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL
```

E.3 FULL Tier, Cohort Sweep ($D = 8$, $h = 2$)

```
# EVICT_SUBTREE_2 (dense, D=8): cohort sweep of a 4-position subtree: 2005 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_1 OP_TOALTSTACK <TAG_NODE> OP_1 OP_ROLL
LEAF_STEP
OP_3 OP_ROLL
LEAF_STEP
HASH_STEP(4)
OP_3 OP_ROLL
LEAF_STEP
OP_5 OP_ROLL
LEAF_STEP
HASH_STEP(6)
HASH_STEP(4)
OP_FROMALTSTACK OP_DROP <EMPTY_2> OP_0
LEVEL  x6
CHECK_OLD
FINAL
```

E.4 FULL Tier, Sparse Profile ($D = 8$)

```
# EXIT_1 (sparse, D=8): 1892 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented
```

```
PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_1 <SEL_OUT_SINGLE> OP_TX
  <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8
  OP_LEFT OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
FINAL

# JOIN_0 (sparse, D=8): 1876 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
FINAL

# JOINK_2 (sparse, D=8): 3263 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_PREV_S
FINAL

# UPDATE (sparse, D=8): 2092 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116
  OP_EQUALVERIFY
LEAF_POS
OP_ROT
LEAF_POS
OP_ROT OP_EQUALVERIFY OP_SWAP
LEAF_POS
SPARSE_SETUP
<TAG_NODE> OP_ROT OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
```

```
FINAL

# CLAIM (sparse, D=8): 3386 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

<SEL_CURR_INPUT> OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_DUP OP_TOALTSTACK OP_SWAP OP_TOALTSTACK
  OP_3 OP_PICK OP_3 OP_PICK OP_CAT OP_CAT
TAGGED(VOUCHER)
OP_3 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGFROMSTACK OP_VERIFY OP_SIZE 116
  OP_EQUALVERIFY OP_SWAP OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_OVER OP_8
  OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB
LE64
OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY
PAD4
OP_CAT OP_FROMALTSTACK OP_ROT OP_TOALTSTACK OP_TOALTSTACK OP_OVER
LEAF_POS
OP_NIP
SPARSE_SETUP
<TAG_NODE> OP_ROT OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
OP_DUP OP_NOT OP_IF OP_SWAP OP_DROP <EMPTY_0> OP_SWAP OP_ENDIF
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_PREV_S
FINAL
```

E.5 FULL Numerics Without Splice or EC Opcodes ($D = 4$)

The intermediate data point of §5.3: BIP-441 numerics kept, while `OP_SUBSTR`, `OP_LEFT`, `OP_RIGHT` and the EC point opcodes are replaced by `OP_CAT`, `OP_SIZE` and `OP_TWEAKADD`.

```
# EXIT_1 (minimal inventory, dense, D=4): 1118 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

<SEL_CURR_INPUT> OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK_MIN
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_DUP
  OP_TOALTSTACK OP_IF OP_3 OP_PICK OP_CHECKLOCKTIMEVERIFY OP_DROP OP_4
  OP_PICK OP_ELSE OP_5 OP_PICK OP_ENDIF OP_CHECKSIGVERIFY OP_SIZE OP_4
  OP_EQUALVERIFY OP_OVER OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_2 OP_PICK
  OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_3 OP_PICK OP_SIZE 32 OP_EQUALVERIFY
  OP_DROP OP_4 OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_5 OP_PICK OP_SIZE
  32 OP_EQUALVERIFY OP_DROP OP_6 OP_PICK OP_SIZE OP_8 OP_EQUALVERIFY OP_DROP
  OP_6 OP_PICK OP_TOALTSTACK OP_5 OP_PICK OP_TOALTSTACK OP_CAT OP_CAT OP_CAT
  OP_CAT OP_CAT OP_CAT
TAGGED(LEAF)
OP_1 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_FROMALTSTACK OP_CAT OP_EQUALVERIFY
  OP_FROMALTSTACK OP_FROMALTSTACK OP_IF OP_2DUP OP_NUMEQUALVERIFY OP_ELSE
  OP_2DUP OP_LESSTHANOREQUAL OP_VERIFY OP_ENDIF OP_NIP <TAG_NODE> OP_ROT
  OP_ROT <EMPTY_0> OP_0
LEVEL_MIN  x4
CHECK_OLD
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_FROMALTSTACK
  OP_DROP OP_1 OP_ELSE OP_SWAP OP_FROMALTSTACK
TREE_SPK_MIN
```

```
OP_0 <SEL_OUT_SINGLE> OP_TX OP_ROT OP_EQUALVERIFY OP_NUMEQUAL OP_ENDIF
```

E.6 MID Tier Macros

```
# Macro definitions, MID tier. Each expansion is itself collapsed
# against the other macros; a macro never contains itself.

## TAGGED(LEAF)
<TAG_LEAF> OP_SWAP OP_CAT OP_SHA256

## TAGGED(NODE)
<TAG_NODE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(STATE)
<TAG_STATE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(VOUCHER)
<TAG_VOUCHER> OP_SWAP OP_CAT OP_SHA256

## TAGGED(POS)
<TAG_POS> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPTWEAK)
<TAG_TAPTWEAK> OP_SWAP OP_CAT OP_SHA256

## PADZ   ([x z w] -> [le32(x)]: z||w == 00000000 and size(x)+size(z) == 4)
OP_OVER OP_SWAP OP_CAT <ZERO4> OP_EQUALVERIFY OP_SIZE OP_ROT OP_SIZE OP_ROT
  OP_ADD OP_4 OP_NUMEQUALVERIFY OP_SWAP OP_CAT

## ENC64   ([x z w] -> [enc64(x)])
PADZ
<00000000_01> OP_CAT

## LE64   ([x z w] -> [le64(x)])
PADZ
<ZERO4> OP_CAT

## TREE_SPK_MIN   ([r m] -> [TREE_SPK(r, m)] with OP_TWEAKADD)
OP_SWAP
TAGGED(STATE)
<NUMS_X> OP_TWEAKADD OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_SWAP OP_TWEAKADD <0x5120> OP_SWAP OP_CAT

## PROLOGUE   (input is #0; (r, m) match the input scriptPubKey; alt <- m r)
32768 OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK_MIN
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK

## LEVEL_MID   (dense level with PADZ gadgets)
OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13
  OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_2 OP_PICK
  OP_TOALTSTACK
ENC64
OP_CAT OP_FROMALTSTACK OP_7 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_7 OP_ROLL
  OP_5 OP_ROLL OP_5 OP_ROLL
ENC64
OP_6 OP_ROLL OP_SWAP OP_CAT OP_2 OP_PICK OP_SWAP OP_4 OP_PICK OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_8 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_5 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_DROP OP_4 OP_ROLL OP_4 OP_ROLL
  OP_4 OP_ROLL
ENC64
OP_3 OP_ROLL OP_SWAP OP_CAT OP_ROT OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_OVER
  OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
```

```
## LEVEL_MID_S   (sparse level: dir folded into the position accumulator)
OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13
  OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_3 OP_ROLL
  OP_FROMALTSTACK OP_SWAP OP_SIZE OP_ONOTEQUAL OP_NOTIF OP_DROP OP_DUP
  OP_ENDIF OP_SWAP <ENC0> OP_CAT OP_DUP OP_CAT
TAGGED(NODE)
OP_FROMALTSTACK OP_FROMALTSTACK OP_7 OP_PICK OP_IF OP_OVER OP_ADD OP_ENDIF
  OP_SWAP OP_DUP OP_ADD OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_3 OP_ROLL OP_3 OP_ROLL OP_3 OP_ROLL OP_2 OP_PICK OP_TOALTSTACK
ENC64
OP_CAT OP_FROMALTSTACK OP_7 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_7 OP_ROLL
  OP_5 OP_ROLL OP_5 OP_ROLL
ENC64
OP_6 OP_ROLL OP_SWAP OP_CAT OP_2 OP_PICK OP_SWAP OP_4 OP_PICK OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_8 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_5 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_DROP OP_4 OP_ROLL OP_4 OP_ROLL
  OP_4 OP_ROLL
ENC64
OP_3 OP_ROLL OP_SWAP OP_CAT OP_ROT OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_OVER
  OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK

## SPARSE_SETUP   ([ok nonce plo phi] -> [] ; alt <- plo 0 1 E_0)
OP_2SWAP OP_CAT
TAGGED(POS)
OP_ROT OP_DUP OP_TOALTSTACK OP_SIZE OP_4 OP_EQUALVERIFY OP_ROT OP_CAT
  OP_EQUALVERIFY OP_0 OP_TOALTSTACK OP_1 OP_TOALTSTACK <EMPTY_0>
  OP_TOALTSTACK

## SPARSE_TEARDOWN   (check the accumulated position against plo; drop bookkeeping)
OP_FROMALTSTACK OP_DROP OP_FROMALTSTACK OP_DROP OP_FROMALTSTACK
  OP_FROMALTSTACK OP_NUMEQUALVERIFY

## CHECK_OLD   (os (padded) == input 0 amount, oh == r)
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
<SEL_IN_AMOUNT> OP_TX OP_EQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_OLD_S   (CHECK_OLD with the sparse teardown)
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
<SEL_IN_AMOUNT> OP_TX OP_EQUALVERIFY
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV   (os == ns_prev, oh == nh_prev; chained steps)
OP_2SWAP OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV_S   (CHECK_PREV with the sparse teardown)
OP_2SWAP
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## LEAF_JOIN   ([amt8 ek ok evk ea nonce seq amt_num za wa] -> [leaf] | amt_num)
OP_2 OP_PICK OP_TOALTSTACK
LE64
OP_7 OP_PICK OP_EQUALVERIFY OP_DUP OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_1
  OP_PICK OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_2 OP_PICK OP_SIZE OP_4
  OP_EQUALVERIFY OP_DROP OP_3 OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_4
  OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_5 OP_PICK OP_SIZE 32
  OP_EQUALVERIFY OP_DROP OP_6 OP_PICK OP_SIZE OP_8 OP_EQUALVERIFY OP_DROP
  OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT

## FINAL   [shown for D = 4; <EMPTY_D> is the empty root of the tree's depth]   (output 0 is
     (le64(ns), TREE_SPK(nh, m)) unless nh == E_D)
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_2DROP
  OP_FROMALTSTACK OP_DROP OP_1 OP_ELSE OP_3 OP_ROLL OP_3 OP_ROLL
LE64
OP_SWAP OP_FROMALTSTACK
```

```
TREE_SPK_MIN
OP_0 <SEL_OUT_SINGLE> OP_TX OP_ROT OP_EQUALVERIFY OP_EQUAL OP_ENDIF
```

E.7 MID Tier, Dense Profile ($D = 4$)

```
# EXIT_1 (MID tier, dense, D=4): 1427 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_3 OP_PICK OP_CHECKLOCKTIMEVERIFY OP_DROP OP_10
  OP_PICK OP_CHECKSIGVERIFY
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_ELSE OP_11 OP_PICK OP_CHECKSIGVERIFY OP_2DROP
  OP_DROP OP_ENDIF OP_1 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_11 OP_PICK OP_CAT
  OP_EQUALVERIFY OP_10 OP_PICK OP_EQUALVERIFY OP_FROMALTSTACK OP_DROP
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_MID  x4
CHECK_OLD
FINAL

# JOIN_0 (MID tier, dense, D=4): 1264 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP
  <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_OLD
FINAL

# JOIN_2 (MID tier, dense, D=4): 952 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP
  <TAG_NODE> OP_ROT OP_ROT <EMPTY_2> OP_0 OP_2SWAP
LEVEL_MID  x2
CHECK_OLD
FINAL

# JOINK_2 (MID tier, dense, D=4): 2005 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP
  <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_OVER OP_SIZE 32 OP_EQUALVERIFY
  OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
FINAL

# UPDATE (MID tier, dense, D=4): 1535 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY 18 OP_PICK OP_CHECKSIGVERIFY
LEAF_JOIN
OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_SWAP
```

```
TAGGED(LEAF)
OP_SWAP <TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
CHECK_OLD
FINAL

# UPDATE_X (MID tier, exit-key consent, dense, D=4): 1539 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
21 OP_PICK OP_CHECKSIGVERIFY <OPERATOR_KEY> OP_CHECKSIGVERIFY 18 OP_PICK
  OP_CHECKSIGVERIFY
LEAF_JOIN
OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_SWAP
TAGGED(LEAF)
OP_SWAP <TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
CHECK_OLD
FINAL

# CLAIM (MID tier, dense, D=4): 2547 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

32768 OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK_MIN
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_DUP OP_TOALTSTACK OP_SWAP OP_TOALTSTACK
  28 OP_PICK 28 OP_PICK 28 OP_PICK 28 OP_PICK 28 OP_PICK 28 OP_PICK 28
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT 19 OP_PICK 19 OP_PICK 19
  OP_PICK 19 OP_PICK 19 OP_PICK 19 OP_PICK 19 OP_PICK OP_CAT OP_CAT OP_CAT
  OP_CAT OP_CAT OP_CAT OP_2 OP_PICK OP_ROT OP_ROT OP_CAT OP_CAT
TAGGED(VOUCHER)
OP_ROT OP_SWAP 27 OP_PICK OP_CHECKSIGFROMSTACK OP_VERIFY OP_DROP 20 OP_PICK
  OP_7 OP_PICK OP_7 OP_ROLL OP_7 OP_ROLL
PADZ
OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_TOALTSTACK
LEAF_JOIN
OP_TOALTSTACK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_DUP OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_7 OP_PICK OP_TOALTSTACK OP_6 OP_PICK
  OP_TOALTSTACK OP_TOALTSTACK OP_6 OP_ROLL OP_DROP OP_7 OP_ROLL OP_6 OP_ROLL
  OP_SUB OP_DUP OP_0 OP_GREATERTHANOREQUAL OP_VERIFY OP_DUP OP_3 OP_ROLL OP_3
  OP_ROLL
LE64
OP_4 OP_ROLL OP_1ADD OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
OP_3 OP_ROLL OP_SWAP OP_CAT OP_CAT
TAGGED(LEAF)
OP_SWAP OP_DUP OP_NOT OP_IF OP_SWAP OP_DROP <EMPTY_0> OP_SWAP OP_ENDIF
  <TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
CHECK_OLD
OP_FROMALTSTACK OP_FROMALTSTACK
TAGGED(LEAF)
OP_SWAP OP_2SWAP OP_SWAP OP_TOALTSTACK OP_TOALTSTACK <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
FINAL
```

E.8 MIN Tier Macros

```
# Macro definitions, MIN tier. Each expansion is itself collapsed
# against the other macros; a macro never contains itself.

## TAGGED(LEAF)
<TAG_LEAF> OP_SWAP OP_CAT OP_SHA256

## TAGGED(NODE)
<TAG_NODE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(STATE)
<TAG_STATE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPTWEAK)
<TAG_TAPTWEAK> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPSIGHASH)
<TAG_TAPSIGHASH> OP_SWAP OP_CAT OP_SHA256

## TAGGED(CHALLENGE)
<TAG_CHALLENGE> OP_SWAP OP_CAT OP_SHA256

## PADZ   ([x z w] -> [le32(x)]: z||w == 00000000 and size(x)+size(z) == 4)
OP_OVER OP_SWAP OP_CAT <ZERO4> OP_EQUALVERIFY OP_SIZE OP_ROT OP_SIZE OP_ROT
  OP_ADD OP_4 OP_NUMEQUALVERIFY OP_SWAP OP_CAT

## LE64   ([x z w] -> [le64(x)])
PADZ
<ZERO4> OP_CAT

## TREE_SPK_MIN   ([r m] -> [TREE_SPK(r, m)] with OP_TWEAKADD)
OP_SWAP
TAGGED(STATE)
<NUMS_X> OP_TWEAKADD OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_SWAP OP_TWEAKADD <0x5120> OP_SWAP OP_CAT

## LEVEL_MID   (dense level with PADZ gadgets)
OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13
  OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_2 OP_PICK
  OP_TOALTSTACK
PADZ
<00000000_01> OP_CAT OP_CAT OP_FROMALTSTACK OP_7 OP_PICK OP_OVER OP_ADD
  OP_TOALTSTACK OP_7 OP_ROLL OP_5 OP_ROLL OP_5 OP_ROLL
PADZ
<00000000_01> OP_CAT OP_6 OP_ROLL OP_SWAP OP_CAT OP_2 OP_PICK OP_SWAP OP_4
  OP_PICK OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_8 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_TOALTSTACK OP_5 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_DROP
  OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
<00000000_01> OP_CAT OP_3 OP_ROLL OP_SWAP OP_CAT OP_ROT OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_OVER OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK

## LEAF_JOIN   ([amt8 ek ok evk ea nonce seq amt_num za wa] -> [leaf] | amt_num)
OP_2 OP_PICK OP_TOALTSTACK
LE64
OP_7 OP_PICK OP_EQUALVERIFY OP_DUP OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_1
  OP_PICK OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_2 OP_PICK OP_SIZE OP_4
  OP_EQUALVERIFY OP_DROP OP_3 OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_4
  OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_5 OP_PICK OP_SIZE 32
  OP_EQUALVERIFY OP_DROP OP_6 OP_PICK OP_SIZE OP_8 OP_EQUALVERIFY OP_DROP
  OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT
```

E.9 MIN Tier, Dense Profile ($D = 4$)

```
# EXIT_1 (MIN tier, dense, D=4): 1700 bytes
# macros in CAPITALS are defined in MACROS_MIN.txt; raw opcodes are wrapped and indented

OP_2DUP
TREE_SPK_MIN
OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_IF OP_3 OP_PICK
  OP_CHECKLOCKTIMEVERIFY OP_DROP OP_10 OP_PICK OP_CHECKSIGVERIFY
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_ELSE OP_11 OP_PICK OP_CHECKSIGVERIFY OP_2DROP
  OP_DROP OP_ENDIF OP_9 OP_PICK 2117922 OP_CAT OP_9 OP_PICK OP_CAT
  OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_MID  x4
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
OP_FROMALTSTACK OP_FROMALTSTACK OP_3 OP_ROLL OP_EQUALVERIFY OP_4 OP_ROLL
  OP_DROP OP_ROT OP_5 OP_ROLL OP_5 OP_ROLL
LE64
OP_3 OP_ROLL OP_FROMALTSTACK
TREE_SPK_MIN
34 OP_SWAP OP_CAT OP_CAT OP_SWAP OP_CAT OP_2 OP_ROLL OP_CAT OP_SHA256 OP_2
  OP_ROLL OP_ROT OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK 34 OP_SWAP OP_CAT
  OP_3 OP_ROLL OP_CAT OP_SHA256 OP_3 OP_ROLL OP_4 OP_ROLL 0 OP_SWAP OP_CAT
  OP_SWAP OP_CAT OP_4 OP_ROLL OP_CAT OP_ROT OP_CAT OP_SWAP OP_CAT OP_3
  OP_ROLL OP_CAT OP_SWAP OP_CAT <0200000000> OP_CAT OP_SWAP OP_CAT
  <00ffffffff> OP_CAT
TAGGED(TAPSIGHASH)
<G_X||G_X> OP_SWAP OP_CAT
TAGGED(CHALLENGE)
OP_OVER 0 OP_CAT OP_EQUALVERIFY <G_X> OP_SWAP OP_CAT OP_1 OP_CAT <G_X>
  OP_CHECKSIG
```